

FeDril

User Acceptance Testing

No-CUI Readiness Workflow

Comprehensive Acceptance Test and Execution Guide

VERSION 2.0

September 12, 2026

Synthetic test data only. This document does not authorize the storage or processing of real CUI and does not represent certification, assessment approval, or government authorization.

Document Contents

Purpose And Scope	4
Out Of Scope	4
Result Vocabulary	4
Acceptance Categories	4
Roles	5
Test Data	6
Pre-Publication Checklist	9
UAT Environment And Execution Rules	9
UAT-01: Confirm No-CUI Mode	10
UAT-02: Verify Role Access Surface	11
UAT-02A: Verify MVP Access-Control Enforcement	11
Tenant Onboarding UAT	12
Profile Module UAT	17
UAT-03: Create The No-CUI Contract Record	19
UAT-04: Upload Contract Document Metadata	20
Contract Deliverables UAT	21
UAT-05: Search Source-Backed Clauses	23
UAT-06: Attach Clauses To Contract	24
UAT-07: Generate And Review Contract Obligations	24
UAT-08: Update Obligation Status	25
UAT-09: Assign Obligation Owner	26
UAT-10: Acknowledge No-CUI Evidence Rules	26
UAT-11: Create Allowed Evidence Metadata	27
UAT-12: Negative Evidence Classification Check	27
CMMC Module UAT	28
UAT-13: Generate Current Report Artifact	32
UAT-14: Verify Audit History For Created Records	32
UAT-15: Verify Audit Access By Role	33
Public Demo And Platform Follow-Up UAT	34
UAT-M01: Submit A Synthetic Public Demo Request	34
UAT-M02: Review And Respond To A Demo Request	34
UAT-M03: Verify Platform Customer List And Detail Administration	35
Identity, Membership, Workspace, And Subscription UAT	36
UAT-I01: Invite, Accept, Resend, Revoke, And Expire A Tenant User	36

UAT-I02: Deactivate A Member And Verify Access Removal	36
UAT-I03: Switch Tenants And Exercise Subscription Lifecycle	37
Dashboard UAT	37
UAT-DB01: Reconcile Executive Dashboard Metrics And Alerts	37
Contract Intelligence And Applicability UAT	38
UAT-X01: Review Extracted Clause Candidates	38
UAT-X02: Evaluate Applicability And Suggested Obligations	39
UAT-X03: Verify SAM Lookup And Contract Size Checks	39
UAT-X04: Review Compliance Content Lifecycle And Development Import Boundary	40
Tasks, Search, Calendar, And Notifications UAT	40
UAT-W01: Create, Search, Complete, Reopen, And Regenerate Tasks	40
UAT-W02: Verify Notification Preferences, Assignments, And Reminders	41
UAT-W03: Create And Complete A Compliance Checklist	42
Extended Evidence And Classification UAT	42
UAT-E01: Upload, Download, Replace, And Delete An Allowed Evidence File	42
UAT-E02: Review Evidence And Enforce Separation Of Responsibility	43
UAT-E03: Complete An Evidence Request Workflow	43
UAT-E04: Verify Classification Review, Classified Notes, And Escalation	44
Extended CMMC, SPRS, And SSP UAT	44
UAT-C05: Create Gaps And Close A POA&M Item	44
UAT-C06: Record Affirmation And Responsibility Assignments	45
UAT-C07: Calculate And Review A Draft SPRS Score	46
UAT-C08: Build, Approve, Package, And Share An SSP Narrative	46
Subcontractor And Partner Collaboration UAT	47
UAT-S01: Create And Review A Subcontractor Profile	47
UAT-S02: Track Flow-Downs And Supplier Obligations	47
UAT-S03: Request Evidence And Generate A Subcontractor Report	48
UAT-S04: Exercise Shared Portal Package Lifecycle	48
SAM.gov SPR Preparation UAT	49
UAT-SPR01: Configure SPR Applicability And Schedule	49
UAT-SPR02: Enter, Import, Review, And Remediate SPR Data	50
UAT-SPR03: Generate, Export, And Record A Manual SPR Receipt	50
Labor Readiness UAT	51
UAT-L01: Record Labor Applicability And Wage Evidence	51
UAT-L02: Classify And Reclassify A Synthetic Employee Assignment	51

UAT-L03: Verify Labor Dashboard And Immutable Report	52
Policies, Suggested Guidance, And Guarded Assistance UAT	52
UAT-AI01: Generate And Review A Policy From A Template	53
UAT-AI02: Verify Citation, Logging, And Fail-Closed Assistant Behavior	53
Extended Reporting And Audit UAT	54
UAT-R01: Verify Report, Export, Archive, Restore, And PDF Lifecycle	54
UAT-R02: Verify Audit Search, Export, Append-Only Behavior, And Rollback	54
No-CUI Governance And Operational Readiness UAT	55
UAT-N01: Verify Contextual Data-Handling Notices And Gating	55
UAT-N02: Exercise CUI-Readiness Checklist And Evidence Without Authorizing CUI	56
UAT-N03: Verify Security, Technical, And Incident Readiness Records	56
UAT-N04: Acknowledge Shared Responsibility And Resolve A Support Escalation	57
UAT-N05: Seed, Verify, And Remove A Synthetic Demo Dataset	57
Enterprise And Regulated-Deployment Boundary UAT	58
UAT-O01: Confirm Future Enterprise Features Do Not Alter MVP Claims Or Access	58
Acceptance Exit Criteria	58
Mandatory Negative And Security Tests	59
Smoke Test Sequence	60
Regression Test Execution	60
API And UI Consistency Checks	61
Evidence Capture Requirements	61
Defect Severity And Triage	61
Requirements Traceability Matrix	61
Test Execution Summary And Sign-Off	62
Known Limitations And Unresolved Gaps	63
Hidden Risks, Edge Cases, And Dependencies	64
Document Change History	66
Status basis: Current UI routes, API endpoints, authorization contracts, services, repositories, migrations, automated tests, recent Git history, and the active working tree were reviewed on 2026-09-12. The original PDF layout and test flow are preserved. Newer functionality is labeled Implemented, Partially implemented, Planned, Blocked by environment, or Do not claim according to available evidence. Do not use real customer CUI, FCI, PHI, classified information, export-controlled or ITAR technical data, credentials, secrets, tokens, payroll records, private keys, proprietary customer documents, or production customer evidence in this UAT. Every record and file used below is synthetic and non-sensitive. Local observation on 2026-09-12 confirmed /health returned 200 and identified the service posture as No-CUI / compliance management only; PostgreSQL, Redis, object storage, background-job coordination, and the configured development ClamAV placeholder reported reachable. The web root returned 200. This is point-in-time local evidence, not production evidence.	

Purpose And Scope

This document provides an executable manual UAT for the complete verified FeDril application at the recorded release candidate. It covers the original onboarding-to-report workflow and newer UI/API features involving authentication, tenant administration, tenant isolation, RBAC, contracts, compliance content, evidence, CMMC, SPRS readiness, SSP, SAM.gov SPR preparation, labor readiness, subcontractors, partner packages, policies, tasks, notifications, reports, exports, audit history, No-CUI governance, and operational readiness.

FeDril is the external product brand. Internal source namespaces, routes, migration names, storage keys, and compatibility identifiers may continue to use GCCS or `Gccs.*`; those internal names are not a second customer-facing product.

Out Of Scope

- Real CUI, FCI, PHI, classified, export-controlled/ITAR, credential, secret, payroll, medical, proprietary customer, or production evidence data.
- A claim or determination that FeDril is CMMC-certified, FedRAMP-authorized, legally compliant, government-approved, audit-ready, or approved to store/process real CUI.
- Legal, accounting, SBA eligibility, labor/wage, assessor, contracting-officer, or government determinations.
- Automated submission or synchronization with SPRS, SAM.gov, CMMC systems, or another external compliance system. SPRS and SPR cases cover readiness, preparation, review, affirmation preparation, export, and user-entered submission metadata only.
- Destructive recovery, denial-of-service, unauthorized penetration, real malware, real identity-provider/key rotation, or regulated-deployment changes.
- Substantive legal accuracy of compliance content; this UAT verifies source, provenance, publication, review, and claim controls.

Result Vocabulary

Label	Meaning
Implemented	Code and relevant tests or UI/API behavior were found; the UAT case must still be executed and evidenced
Partially implemented	Some workflow layers exist, but UI, provider, persistence, lifecycle, or verification is incomplete
Planned	Documented future behavior that is not a current acceptance target
Blocked by environment	Execution cannot complete because a required dependency, configuration, migration, provider, build, or approved environment is unavailable
Not applicable	Outside the approved release/MVP; requires a reason and approver
Failed	An expected result was not observed; record a defect and apply blocker rules
Do not claim	The behavior or evidence does not support certification, authorization, legal, government, external-submission, or real-CUI claims

Acceptance Categories

Category	UAT Coverage	Current-State Label
Pilot and paid tenant onboarding	UAT-T01 through UAT-T04	Implemented for pending provisioning and Owner activation; paid billing verification is partially implemented
Company profile	UAT-P01 through UAT-P04	Implemented
One configured No-CUI readiness workflow that shows contract metadata	UAT-01 through UAT-04	Implemented
Contract deliverables	UAT-D01 through UAT-D04	Implemented
Attached or reviewed clauses	UAT-05 through UAT-06	Implemented
Generated obligations	UAT-07	Implemented
Owner/status tracking	UAT-08 through UAT-09	Implemented
Allowed evidence metadata	UAT-10 through UAT-12	Implemented
CMMC readiness module	UAT-C01 through UAT-C04	Implemented
A current report artifact	UAT-13	Implemented
Audit history	UAT-14 through UAT-15	Implemented

Category	UAT Coverage	Current-State Label
Public demo, platform follow-up, and customer administration	UAT-M01 through UAT-M03	Implemented; external email and HubSpot delivery are environment-dependent
Membership, invitation, tenant selection, and subscriptions	UAT-I01 through UAT-I03	Implemented; production identity/session behavior requires a production-like identity provider
Executive dashboard	UAT-DB01	Implemented
Contract extraction, applicability, size checks, content governance, and expert review	UAT-X01 through UAT-X04	Implemented; SAM.gov and background-worker behavior are environment-dependent
Tasks, checklists, search, notifications, and reminders	UAT-W01 through UAT-W03	Implemented; email delivery is environment-dependent
Evidence files, reviews, requests, and classification	UAT-E01 through UAT-E04	Implemented; malware-scanner behavior requires the configured scanner
Extended CMMC, SPRS, responsibility, and SSP workflows	UAT-C05 through UAT-C08	Implemented as internal readiness workflows; external AI is disabled
Subcontractor, supplier obligation, and partner portal workflows	UAT-S01 through UAT-S04	Implemented; external portal identity/package catalogs are partially implemented
SAM.gov Subcontracting Plan Reporting preparation	UAT-SPR01 through UAT-SPR03	Implemented for preparation and user-entered receipts; automated submission is unavailable and must not be claimed
Labor readiness	UAT-L01 through UAT-L03	Implemented; focused labor dashboard/report, redaction, authorization, and migration checks pass; PostgreSQL execution remains environment-gated.
Policy and guarded-assistance workflows	UAT-AI01 through UAT-AI02	Implemented as draft, cited, human-reviewed workflows; external AI is disabled
Report export, PDF lifecycle, and audit integrity	UAT-R01 through UAT-R02	Implemented; renderer, storage, and PostgreSQL transaction evidence are environment-dependent
No-CUI notices, readiness evidence, escalations, incident-readiness records, and synthetic demo lifecycle	UAT-N01 through UAT-N05	Implemented as readiness/governance workflow only; does not authorize real CUI
Enterprise and regulated-deployment scaffolding	UAT-O01	Planned or partially implemented; not applicable to No-CUI MVP acceptance and must not be represented as authorization

Roles

Role	UAT Actor	Use In This Test	Permission Expectation
Platform Operator	Casey Morgan, casey.morgan+platform-uat@example.com	Internal Pilot and Paid tenant onboarding	Can open the platform console and provision, list, resend, or cancel pending onboarding only when assigned ProvisionTenants or Gccs.PlatformOperator
Pilot Tenant Owner	Riley Chen, riley.chen+pilot-uat@example.com	Accept the Pilot Owner invitation	Becomes Owner only after accepting the invitation with the exact invited email
Paid Tenant Owner	Taylor Reed, taylor.reed+paid-uat@example.com	Accept the Paid Owner invitation	Becomes Owner only after accepting the invitation with the exact invited email
Owner	Morgan Lane, morgan.lane+uat@example.com	Tenant setup, No-CUI mode, audit review	Can manage the profile, contracts, deliverables, tenant settings, and audit review
Admin	Jordan Miles, jordan.miles+uat@example.com	User setup and operational fallback	Can manage the profile, contracts, deliverables, users, and workflow records, but not tenant ownership

Role	UAT Actor	Use In This Test	Permission Expectation
Compliance Manager	Priya Shah, priya.shah+uat@example.com	Profile, contract, deliverable, clause, obligation, evidence, reports	Can manage the core compliance workflow, including the profile and contract deliverables
Contributor	Devin Brooks, devin.brooks+uat@example.com	Evidence metadata and assigned work	Can view the profile, contracts, and deliverables; can manage evidence and tasks; cannot modify profile or contract deliverables
Auditor	Elena Carter, elena.carter+uat@example.com	Read-only acceptance review	Can view the profile, contracts, deliverables, and reports; cannot modify workflow data or audit logs
Advisor	Avery Quinn, avery.quinn+advisor@example.com	External compliance advisor review	Can view but not modify the profile; can manage contracts and deliverables and view audit history inside the tenant boundary

Note: In local development, the React app uses development authentication headers. Treat the names above as test personas unless separate login accounts are configured. Switch user lists only active users with an active membership in the selected tenant. Before UAT-09, confirm Priya Shah and Devin Brooks have active memberships; an invitation that has not been accepted is not sufficient.

Tenant-onboarding prerequisite: Tenant onboarding is an internal platform-operations page, not a tab inside a customer tenant. A normal tenant Owner or a user with ManageTenant cannot provision tenants. In staging or production, use an approved internal account assigned the Gccs.PlatformOperator application role. For local development, start the web app with VITE_GCCS_DEV_PLATFORM_PERMISSIONS=ProvisionTenants before opening /platform; the default local configuration does not grant this permission.

Test Data

Data Type	Field	Value
Pilot onboarding	Customer reference	PILOT-UAT-026-A
Pilot onboarding	Tenant display name	Blue Ridge Pilot Workspace - Synthetic
Pilot onboarding	Pilot end date	2027-01-31
Pilot onboarding	Setup reason	Provision approved synthetic No-CUI pilot for UAT; no customer files.
Pilot onboarding	Owner email	riley.chen+pilot-uat@example.com
Pilot onboarding	Owner display name	Riley Chen
Paid onboarding	Customer reference	CUSTOMER-UAT-026-A
Paid onboarding	Tenant display name	Blue Ridge Paid Workspace - Synthetic
Paid onboarding	Plan code	FOUNDATION-ANNUAL
Paid onboarding	Subscription reference	SUB-UAT-00026-A
Paid onboarding	Setup reason	Provision approved synthetic No-CUI paid workspace for UAT; billing reference checked manually.
Paid onboarding	Owner email	taylor.reed+paid-uat@example.com
Paid onboarding	Owner display name	Taylor Reed
Cancellation fixture	Customer reference	PILOT-UAT-CANCEL-026-A
Cancellation fixture	Tenant display name	Blue Ridge Cancelled Pilot - Synthetic
Cancellation fixture	Cancellation reason	Duplicate synthetic onboarding created for cancellation UAT.
Tenant	Active tenant	GCCS Development Tenant
Tenant mode	Data handling mode	NoCui
Profile	Legal entity	Blue Ridge Federal Support LLC
Profile	DBA	Blue Ridge Support
Profile	UEI	UAT123ABC456
Profile	CAGE	7UAT1

Data Type	Field	Value
Profile	SAM expires	2027-07-31
Profile	Primary NAICS	541512 - Computer Systems Design Services
Profile	NAICS size basis	UAT synthetic value - not an SBA determination
Profile	NAICS status	Small for this synthetic fixture only
Profile	Agency customers	DHS synthetic UAT customer
Profile	Contractor role	Subcontractor
Profile	Products and services	Synthetic non-CUI help desk and compliance support services.
Profile	Employees	Small
Profile	Revenue	Small
Profile	Location	UAT Headquarters
Profile	Address	100 Test Plaza, Arlington, VA, 22201, USA
Profile	IT summary	Synthetic Microsoft 365 and managed endpoint environment used only for No-CUI UAT.
Profile	FCI/CUI posture	FCI only
Profile	Key systems	Microsoft 365, synthetic help desk
Contract	Contract number	DEMO-NC-26-0007
Contract	Title	Non-CUI Help Desk Support BPA Call
Contract	Agency or prime	Fictional Prime Systems Inc. for DHS
Contract	Role	Subcontractor
Contract	Contract type	Fixed price
Contract	Status	Active
Contract	Awarded	2026-06-15
Contract	Start	2026-07-01
Contract	End	2027-06-30
Contract	FCI/CUI posture	FCI only
Contract	Place of performance	Virginia, remote support
Contract	Description	Synthetic No-CUI contract for UAT. FCI-only support workflow, no customer CUI, no classified data, no export-controlled technical data.
Deliverable	Name	Monthly service status report - synthetic
Deliverable	Owner	Contracts
Deliverable	Due date	2026-08-31
Deliverable	Initial status	Not started
Deliverable	Description	Synthetic monthly performance summary containing no customer CUI or sensitive government data.
Deliverable negative test	Name	Overdue corrective-action summary - synthetic
Deliverable negative test	Owner	Compliance
Deliverable negative test	Due date	2026-07-15
Deliverable negative test	Status	In progress
Document	File name	demo-nc-contract.txt
Document	Content	Synthetic No-CUI contract fixture for UAT. Includes FAR 52.204-21, FAR 52.204-25, and FAR 52.204-27 references. No customer CUI, classified information, export-controlled technical data, credentials, payroll, or secrets.
Clause	Search terms	52.204-21, 52.204-25, 52.204-27
Evidence	Title	MFA configuration summary - synthetic
Evidence	Type	System configuration

Data Type	Field	Value
Evidence	Owner	Security
Evidence	Status	Approved
Evidence	Effective	2026-06-01
Evidence	Expires	2027-01-31
Evidence	Tags	FAR 52.204-21, FCI, MFA, UAT
Evidence	Controls	AC.L1-3.1.1
Evidence	Classification	FCI
Evidence	Classification reason	User confirmed synthetic FCI-only evidence for No-CUI UAT.
Report	Package title	Prime review evidence package - No-CUI UAT
CMMC	Assessment name	No-CUI Level 1 readiness workspace
CMMC	Target level	Level 1
CMMC	Framework	FAR basic safeguarding
CMMC	Status	In progress
CMMC	Started	2026-06-15
CMMC	Affirmation due	2027-06-15
CMMC	Owner	Security
CMMC	Control for readiness review	AC.L1-3.1.1
CMMC POA&M	Control	AC.L1-3.1.1
CMMC POA&M	Risk	High
CMMC POA&M	Status	Open
CMMC POA&M	Owner	Security
CMMC POA&M	Due date	2026-07-15
CMMC POA&M	Gap	Synthetic UAT gap: document annual access review evidence.
CMMC POA&M	Remediation plan	Upload synthetic access review summary and link it to the control.
Run control	Run ID	UAT-20260912-A; increment the suffix for each rerun
Tenant A	Display name	Blue Ridge Federal Support - UAT A
Tenant B	Display name	Cascade Federal Services - UAT B
Tenant B	Contract number	TENANT-B-NC-0001
Public demo	Contact	Jamie Tester,jamie.testers+demo-uat@example.com,555-010-2026
Public demo	Company and need	Blue Ridge Demo Company - Synthetic; Evaluate No-CUI workflow using synthetic records only.
Task	Title	Review synthetic MFA evidence
Task	Due dates	T+14 for normal work; T-1 for overdue validation
Notification	Preference	Assignment email enabled; due-date digest enabled at 09:00 America/New_York
Evidence file	Allowed file	uat-mfa-summary.txt, UTF-8 plain text containing Synthetic FCI-only MFA configuration summary. No customer data.
Evidence file	Replacement file	uat-mfa-summary-v2.txt, same safe content plus Revision 2.
Evidence negative	Prohibited classification label	Cui; use the harmless allowed bytes above and never real CUI
Evidence request	Title	Synthetic annual access-review evidence request
Classified note	Allowed text	Synthetic FCI-only note for UAT; contains no customer or operational data.
CUI escalation	Summary	Synthetic classification uncertainty; no real sensitive content was supplied.
Subcontractor	Name	Potomac Synthetic Services LLC
Subcontractor	UEI/CAGE	SUBUAT123456 / 9UAT9
Subcontractor	Contact	Alex Partner,alex.partner+uat@example.com

Data Type	Field	Value
Flow-down	Clause/status	FAR 52.204-21; Sent, then Acknowledged
Supplier obligation	Title	Provide annual synthetic safeguarding evidence
Portal package	Title	Synthetic prime-review package - UAT
SPR applicability	Reporting role/period	Individual plan; FY2027-H1
SPR report row	Category/amount	Small business; 125000 whole synthetic dollars
SPR identity	UEI/PIID	UAT123ABC456 / DEMO-NC-26-0007
SPR import	File	spr-uat-valid.csv; formula-like cells and real financial data are prohibited
SPR receipt	Reference	SAM-SPR-UAT-RECEIPT-001; user-entered synthetic metadata only
Labor applicability	Scope	SCA, source https://example.invalid/uat/labor-source, rationale Synthetic applicability review only.
Labor category	Title/rates	Help Desk Specialist - Synthetic; wage 32.50, fringe 4.75
Labor employee	Identifier	UAT-EMP-0001; display name/email are synthetic and visible only to authorized sensitive-data roles
Labor assignment	Effective period	T through T+180
Policy	Template/output	Access Control Policy - Synthetic; draft text with placeholders completed using synthetic facts
Suggested obligation	Summary	Review synthetic access-control evidence annually.
SSP	Section/title	3.1 Access Control; Synthetic SSP narrative
SSP package	Title	Internal SSP review package - synthetic
Readiness evidence	Source	Synthetic restore rehearsal record; URL https://example.invalid/uat/restore-evidence
Security readiness	Review note	Synthetic control verification for UAT; no production security details.
Report	PDF/export title	UAT No-CUI readiness snapshot

Pre-Publication Checklist

Before using this UAT as a sales, demo, or customer-facing asset, confirm:

Check	Required Evidence
Does the UI expose this flow?	The platform Overview and Tenant onboarding navigation items, Owner invitation-acceptance page, and tenant Dashboard, Profile, Settings, Contracts, Calendar, Obligations, Evidence, CMMC, and Reports tabs are visible for the role under test
Does the API enforce this rule?	Endpoint returns expected success, 403, 404, or validation error
Is there a test proving it?	Relevant API test exists for the behavior
Does wording avoid overclaims?	No claim of certification, official compliance status, legal advice, government approval, guaranteed security, or audit-ready status
Does it preserve No-CUI posture?	Test data is synthetic and NoCui mode rejects CUI handling

UAT Environment And Execution Rules

Application prerequisites:

1. Record the exact commit SHA, branch, working-tree state, deployment ID, API URL, web URL, database provider, object-storage mode, scanner mode, identity-provider mode, background-worker mode, and current compliance-content revision.
2. Use dedicated Tenant A and Tenant B workspaces. Do not run destructive or cross-tenant tests in a customer tenant.
3. Apply all required database migrations to a disposable UAT database and retain clean-install and upgrade evidence.
4. Use separate single-role accounts. Capture the top-level permissions array from GET /api/me/access; do not infer active

permissions from `rolePermissionMatrix`.

5. Confirm `/health` returns 200 and reports dependency state. A healthy response proves only point-in-time reachability.
6. Confirm testers have an approved API client, supported browser, clean browser profiles, export/PDF viewer, log/trace access, and a safe evidence location.

Responsibilities:

- The UAT Lead controls scope, sequencing, evidence completeness, defects, retests, and the go/no-go recommendation.
- Product confirms current acceptance scope and resolves documentation conflicts.
- Engineering supplies deployment, configuration, trace, migration, worker, and rollback evidence without exposing secrets.
- Security reviews tenant isolation, RBAC, No-CUI, file, export, audit, and incident-readiness results.
- Compliance-content reviewers validate source, review-state, effective-date, and claim language. UAT does not determine substantive legal correctness.
- Operations supplies health, monitoring, malware-scanner, backup/restore, retention, and provider-failure evidence.

Test execution rules:

1. Execute tests in ID order unless a prerequisite states otherwise.
2. For every mutation, capture the before state, action, response, refreshed after state, and relevant audit event. A toast message alone is not evidence.
3. For rejected, duplicate, stale, or cross-tenant actions, verify no unauthorized business record, successful audit event, job, notification, token, file, or external side effect was created.
4. Record one of `Pass`, `Fail`, `Blocked by environment`, `Not applicable`, or `Not run`. `Implemented` is not a test result.
5. Mark `Not applicable` only with a written reason and UAT Lead approval. Planned enterprise features are not silently promoted into MVP acceptance.
6. Stop execution and notify Security for cross-tenant disclosure, authentication bypass, real sensitive data, exposed secrets/tokens, or an uncontrolled file upload.
7. Redact authorization headers, cookies, signed URLs, tokens, personal data, file contents, and secret values from evidence.

Standard execution record used by every test:

Status: `Pass` / `Fail` / `Blocked by environment` / `Not applicable` / `Not run` | Defect: _____ |
 Tester: _____ | Date/time: _____ | Tenant: _____ | Evidence location: _____ | Notes:

UAT-01: Confirm No-CUI Mode

Category: One configured No-CUI readiness workflow that shows contract metadata.

Role: Owner.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Settings`.

Steps:

1. Open the FeDril app in local development.
2. Click the `Settings` tab.
3. Find `Data handling mode`.
4. Confirm the displayed mode is `NoCui`.
5. If the form is editable, enter `NoCui` for `Mode`.
6. Enter `UAT reset to No-CUI compliance management mode. for Reason for mode change`.
7. Leave `Approval checklist ID` blank.
8. Click `Update mode`.
9. Confirm `Tenant data handling mode history` shows a row with `New = NoCui`.

Expected result: The tenant is configured for No-CUI compliance management only.

Reason: This establishes the safety boundary for every later step. If the tenant is not in `NoCui`, evidence, reporting, and contract-document expectations may not match the MVP posture.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-02: Verify Role Access Surface

Category: One configured No-CUI readiness workflow that shows contract metadata.

Role: Owner or Admin.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tab: Settings.

Steps:

1. Stay on Settings.
2. Review the visible role guidance or current access details if shown.
3. Confirm the test actor can access Contracts, Obligations, Evidence, and Reports.
4. Sign in as Auditor, open Reports, click an existing report card, and confirm the full report detail is brought into view and can be read.
5. Confirm a direct tenant-scoped GET /api/reports/{reportId} for that same report returns 200, while a cross-tenant request returns 404.
6. Confirm all report-generation and archive controls are absent for Auditor, and that direct POST attempts to every report-generation endpoint and /api/reports/{reportId}/archive return 403 without changing the report, creating an audit event for the report, or causing another side effect.
7. Confirm Contributor can help with evidence and task work but cannot generate reports or approve evidence.

Expected result: UAT actors are assigned to roles that match the work they perform.

Reason: A UAT result is not meaningful if the actor has excessive privileges. This step separates operator behavior from reviewer behavior.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-02A: Verify MVP Access-Control Enforcement

Category: Authorized users and authorized transactions/functions.

Role: Owner or Admin for setup; Auditor and Compliance Manager for role checks.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tabs: CMMC, Reports, and Settings.

Current-state label: Implemented for tenant-scoped API permission gates; partially implemented as UI affordances.

Implementation evidence: Protected API routes require authentication, active tenant context, and endpoint permissions such as ViewCmmc, ManageCmmc, ViewReports, and ManageReports. Tenant membership authorization is enforced outside local

development. Local development uses explicit development authentication headers and should not be treated as production identity proof.

Steps:

1. Sign in or switch local development context to Auditor.
2. Click the CMMC tab.
3. Confirm CMMC readiness data can be viewed.
4. Confirm create/update controls such as Create assessment, Save assessment, Create POA&M, or equivalent mutation controls are unavailable or disabled for Auditor.
5. Click the Reports tab.
6. Confirm existing report cards can be opened and read.
7. Confirm report generation and archive controls are unavailable or disabled for Auditor.
8. If an authorized API harness is available, call GET /api/me/access as the Auditor and inspect the top-level permissions array for the active user. Confirm that permissions includes ViewCmmc and ViewReports, but does not include ManageCmmc or ManageReports.
 - Do not use rolePermissionMatrix for this pass/fail check. That matrix lists permissions for every role in the system, so it can contain ManageCmmc and ManageReports even when the active Auditor does not have those permissions.
 - Expected Auditor pass condition: the response may include rolePermissionMatrix.Owner, rolePermissionMatrix.Admin, rolePermissionMatrix.Compliance Manager, or rolePermissionMatrix.Advisor entries with management permissions, but the top-level permissions array for the active Auditor must not contain ManageCmmc or ManageReports.
 - If ManageCmmc or ManageReports appears in the top-level permissions array, the request is not using a clean Auditor context. Confirm roles contains only Auditor, remove any X-Gccs-Dev-Permissions header from Postman, and confirm the local development selector is set to Auditor.
9. If an authorized API harness is available, attempt:
 - POST /api/cmmc/assessments.
 - PATCH /api/cmmc/assessments/{assessmentId}/controls/{controlId}.
 - POST /api/reports/cmmc-readiness?assessmentId={assessmentId}.
10. Confirm each unauthorized mutation returns 403 and does not create, update, archive, or generate records.
11. Switch local development context to Compliance Manager or Owner.
12. Confirm the same CMMC/report actions are available when the role has the required permissions.
13. If an authorized API harness is available, repeat GET /api/me/access and confirm the response includes the permissions required for the action under test.
14. If tenant A and tenant B test contexts are available, attempt to access a tenant B resource while sending tenant A context.
15. Confirm the cross-tenant request returns 403 or 404 and does not disclose tenant B data.
16. Return to Settings, find Audit log, and confirm successful allowed mutations create audit events. Do not expect audit events for every rejected authorization attempt unless the endpoint explicitly records them.

Expected result: Signed-in users can access only tenant records and application functions authorized by their active tenant role. Direct API calls are rejected when the role lacks the required permission.

Reason: This verifies the MVP implementation of "authorized users" and "authorized transactions/functions" without claiming full CMMC certification or complete customer control satisfaction.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Tenant Onboarding UAT

Current-state label: Implemented for internal platform-operator provisioning, pending Owner invitations, invitation acceptance, and tenant activation. Partially implemented for paid commercial lifecycle because FeDril records an operator-confirmed subscription reference but does not call a billing provider.

Implementation evidence: The internal route /platform/tenants/new calls platform-scoped endpoints under /api/platform. The API requires ProvisionTenants or the Gccs.PlatformOperator application role, creates Pilot and Paid tenants in NoCui, keeps them PendingActivation until the invited Owner accepts, records audit and mode-history entries, and protects retries with an idempotency key. Focused tests cover platform authorization, Pilot creation, Paid-field validation,

duplicate requests, cancellation, and Pilot Owner activation. Invitation email delivery requires separately configured provider infrastructure.

Local setup for these cases:

1. Stop any existing local FeDril web process that was started without platform permissions.
2. From the repository root, start the local stack with:

```
VITE_GCCS_DEV_PLATFORM_PERMISSIONS=ProvisionTenants npm run dev
```

3. Open `http://localhost:5173/platform`.
4. Confirm the Overview page loads and the platform navigation shows Tenant onboarding.
5. If Provisioning access denied appears, stop and restart the web app with the environment variable above. Do not substitute a tenant Owner, Admin, or ManageTenant permission.
6. Before repeating these cases, change the final synthetic reference suffix from A to B, C, or another unused value. Customer and subscription references are intentionally unique.

UAT-T01: Create A Pending Pilot Tenant

Category: Pilot and paid tenant onboarding.

Role: Platform Operator.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Navigation: Platform operations -> Tenant onboarding.

Page: Tenant onboarding.

Form sections: Onboarding type, Tenant record, Initial Owner, and Operator confirmations.

Steps:

1. On the platform Overview page, click Tenant onboarding.
2. Confirm the page shows Signed in as with the Platform Operator identity.
3. Confirm the No-CUI product boundary band states that tenant creation does not authorize CUI, classified, ITAR, or sensitive government data.
4. Under Pending tenant onboardings, confirm the list loads or shows No tenant onboardings are awaiting Owner acceptance.
5. In Onboarding type, select Pilot.
6. Confirm Pilot end date is visible and Plan code, Subscription reference, and Commercial approval confirmed are not visible.
7. In Tenant record, enter:
 - Customer reference: PILOT-UAT-026-A.
 - Tenant display name: Blue Ridge Pilot Workspace - Synthetic.
 - Pilot end date: 2027-01-31.
 - Setup reason: Provision approved synthetic No-CUI pilot for UAT; no customer files.
8. In Initial Owner, enter:
 - Owner email: riley.chen+pilot-uat@example.com.
 - Owner display name: Riley Chen.
9. In Operator confirmations, select No-CUI boundary confirmed.
10. Record the displayed Request key without changing it.
11. Click Create pending tenant once.
12. Confirm the success panel heading is Blue Ridge Pilot Workspace - Synthetic and its kicker says Pilot onboarding created.
13. Confirm the labeled values show:
 - Customer reference: PILOT-UAT-026-A.
 - Onboarding status: PendingOwnerAcceptance.
 - Tenant status: PendingActivation.
 - Invitation: Pending.
 - Email delivery: Queued or Sent.
 - Data handling: NoCui.

14. Record `Tenant ID` and `Onboarding ID`. Do not record or request an invitation token.
15. Confirm the pending-onboarding list contains the tenant display name, customer reference, Owner email, and delivery status.

Expected result: One synthetic Pilot onboarding is created as a pending NoCui tenant with a pending Owner invitation. No customer user or membership becomes active before invitation acceptance. Queued passes pending creation; invitation activation cannot be tested until delivery reaches `Sent` and the activation link is available.

Reason: Pilot onboarding is a time-bound commercial path, but it does not weaken the No-CUI boundary or activate the tenant before the invited Owner proves control of the invited identity.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-T02: Create A Pending Paid Tenant

Category: Pilot and paid tenant onboarding.

Role: Platform Operator.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Navigation: Platform operations -> Tenant onboarding.

Page: Tenant onboarding.

Form sections: Onboarding type, Tenant record, Initial Owner, and Operator confirmations.

Steps:

1. If the Pilot success panel is still displayed, click `Provision another tenant`.
2. In `Onboarding type`, select `Paid`.
3. Confirm `Plan code`, `Subscription reference`, and `Commercial approval confirmed` appear, and `Pilot end date` is removed.
4. In `Tenant record`, enter:
 - Customer reference: `CUSTOMER-UAT-026-A`.
 - Tenant display name: `Blue Ridge Paid Workspace - Synthetic`.
 - Plan code: `FOUNDATION-ANNUAL`.
 - Subscription reference: `SUB-UAT-00026-A`.
 - Setup reason: `Provision approved synthetic No-CUI paid workspace for UAT; billing reference checked manually`.
5. In `Initial Owner`, enter:
 - Owner email: `taylor.reed+paid-uat@example.com`.
 - Owner display name: `Taylor Reed`.
6. Select `No-CUI boundary confirmed`.
7. Leave `Commercial approval confirmed` clear and click `Create pending tenant`.
8. Confirm the browser does not submit the form and identifies the missing required confirmation.
9. Select `Commercial approval confirmed` only for this synthetic UAT after confirming the subscription reference matches the test data above.
10. Record the displayed `Request key` and click `Create pending tenant` once.
11. Confirm the success panel heading is `Blue Ridge Paid Workspace - Synthetic` and its kicker says `Paid onboarding created`.
12. Confirm the labeled values show:
 - Customer reference: `CUSTOMER-UAT-026-A`.
 - Onboarding status: `PendingOwnerAcceptance`.
 - Tenant status: `PendingActivation`.
 - Invitation: `Pending`.

- Email delivery: Queued or Sent.
- Data handling: NoCui.

- Record Tenant ID and Onboarding ID without recording an invitation token.
- Confirm the pending-onboarding list contains the paid tenant and labels it Paid.

Expected result: One synthetic Paid onboarding is created as a pending NoCui tenant only after the paid-only fields and commercial confirmation are present. This confirms that FeDril records the operator's commercial approval; it does not prove payment, subscription validity, renewal, cancellation, or billing-provider synchronization.

Reason: Paid status changes the commercial onboarding fields, not the product's data-handling authorization.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-T03: Accept Pilot And Paid Owner Invitations

Category: Pilot and paid tenant onboarding.

Roles: Pilot Tenant Owner and Paid Tenant Owner.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Page: FeDril account activation, opened from the single-use invitation link.

Form: Tenant invitation or the tenant display name.

Environment dependency: Invitation delivery must be configured and the Owner must have the actual activation link. If delivery remains Queued or Failed, record this case as Blocked by environment; do not retrieve a token from the database or ask the Platform Operator to disclose one.

Steps:

Repeat these steps once for the Pilot invitation and once for the Paid invitation:

- Confirm the Platform Operator result shows Email delivery = Sent. If it shows Failed, correct the provider configuration before using Resend invitation; do not resend while delivery is still Queued.
- Open the activation link from the synthetic Owner mailbox.
- In staging or production, sign in through Microsoft Entra using the exact invited email address.
- In local development, if the page shows Use invited test identity, enter the exact value in Invited email and click Continue as invitee:
 - Pilot: riley.chen+pilot-uat@example.com.
 - Paid: taylor.reed+paid-uat@example.com.
- Confirm the activation page shows the expected tenant display name.
- Confirm the summary labels show Account = the invited email and Role = Owner.
- In Display name, enter Riley Chen for Pilot or Taylor Reed for Paid.
- Click Accept invitation.
- Confirm the page shows Workspace activated and an Open workspace link.
- Click Open workspace and confirm the activated tenant is selected for the current browser.
- Confirm the Pilot tenant status is Trialing and the Paid tenant status is Active. If status is not visible in the workspace selector, use an authorized GET /api/me/tenants request and match the recorded Tenant ID.
- Confirm the active user has one Owner membership for the matching tenant and cannot see the other synthetic tenant unless separately invited.
- Reopen the same activation link and confirm it is unavailable because the invitation is already accepted.

Expected result: The exact invited identity accepts each single-use invitation. Pilot activation produces a Trialing tenant; Paid activation produces an Active tenant. The accepted Owner receives only the matching tenant membership.

Reason: Tenant creation and tenant activation are separate security events. The initial Owner must prove control of the invited email before FeDril creates the active membership.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-T04: Verify Onboarding Authorization, Safe Retry, And Cancellation

Category: Pilot and paid tenant onboarding.

Roles: Platform Operator and a normal tenant Owner.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Navigation: Platform operations -> Tenant onboarding.

Steps:

1. Sign in as a normal tenant Owner who does not have ProvisionTenants or Gccs.PlatformOperator.
2. Open /platform/tenants/new directly.
3. Confirm the page shows Provisioning access denied and does not show Create pending tenant.
4. If an authorized API harness is available, call POST /api/platform/tenants with only tenant ManageTenant; confirm it returns 403 and creates no tenant, invitation, onboarding, mode-history, or audit record.
5. Return to the Platform Operator context and create a disposable Pilot record using:
 - Customer reference: PILOT-UAT-CANCEL-026-A.
 - Tenant display name: Blue Ridge Cancelled Pilot - Synthetic.
 - Pilot end date: 2027-01-31.
 - Setup reason: Create synthetic pending tenant for cancellation UAT.
 - Owner email: cancelled.owner+uat@example.com.
 - Owner display name: Cancelled UAT Owner.
 - No-CUI boundary confirmed: selected.
6. Do not accept this invitation.
7. Under Pending tenant onboardings, locate Blue Ridge Cancelled Pilot - Synthetic.
8. Click the cancel icon whose accessible name is Cancel onboarding for Blue Ridge Cancelled Pilot - Synthetic.
9. Confirm Confirm cancellation is disabled while Cancellation reason is empty.
10. Enter Duplicate synthetic onboarding created for cancellation UAT. in Cancellation reason.
11. Click Confirm cancellation.
12. Confirm the record disappears from the pending list.
13. If an authorized API harness is available, confirm the preserved record has onboarding status Cancelled, tenant status Archived, invitation status Revoked, and email delivery Cancelled.
14. Confirm the revoked activation link cannot be accepted.
15. Verify the audit history records the Platform Operator, cancellation reason, timestamp, onboarding transition, and invitation revocation. Do not expect the platform page itself to display tenant audit entries.
16. For API idempotency verification, repeat a successful provisioning request with the same Idempotency-Key and identical payload; confirm it returns the original tenant with 200 and isReplay = true.
17. Reuse that same key with a changed display name; confirm it returns 409 Tenant provisioning conflict and creates no second tenant.
18. Submit a new request key with a previously used customer reference or paid subscription reference; confirm it returns 409 and does not disclose unrelated tenant details.

Expected result: Customer tenant permissions cannot reach platform provisioning. Identical retries are idempotent, conflicting or duplicate requests are rejected, and only a still-pending onboarding can be cancelled with a recorded reason. Activated onboarding is not cancellable through this operation.

Reason: These controls prevent customer self-provisioning, duplicate tenants, unsafe retries, and deletion of compliance-relevant onboarding history.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record,

success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Profile Module UAT

Current-state label: Implemented.

Implementation evidence: The Profile tab calls tenant-scoped GET `/api/company-profile` and PUT `/api/company-profile`. The API requires `ViewCompanyProfile` to read and `ManageCompanyProfile` to save. Focused tests cover draft persistence, completion validation, completion percentage, tenant isolation, and audit history.

UAT-P01: Save An Incomplete Company Profile Draft

Category: Company profile.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it Passed.

Tab: Profile.

Steps:

1. Click the Profile tab in the left navigation.
2. Confirm the page heading is `Create company profile` if no profile exists, or shows the current legal entity name if a profile already exists.
3. If an existing profile contains production or customer data, stop and use a dedicated UAT tenant. Do not overwrite customer data.
4. Enter `Blue Ridge Federal Support LLC` in `Legal entity`.
5. Leave `UEI`, `CAGE`, and `SAM expires` blank for this negative validation step.
6. Click `Save draft`.
7. Confirm the status message is `Draft saved..`
8. Confirm the completion meter shows `Draft` and a value below 100%.
9. Reload the browser.
10. Return to the Profile tab.
11. Confirm `Blue Ridge Federal Support LLC` remains in `Legal entity` and the profile still shows `Draft`.

Expected result: The incomplete profile persists as a draft and is not represented as complete.

Reason: Users need to preserve partial onboarding work without bypassing the server-side fields required for profile completion.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-P02: Verify Completion Validation

Category: Company profile.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it Passed.

Tab: Profile.

Steps:

1. Stay on Profile with the incomplete draft from UAT-P01.
2. Click `Complete` profile without entering the missing required values.
3. Confirm an error summary appears instead of a success message.
4. Confirm the summary identifies missing completion data, including `uei`, `cageCode`, and `samRegistrationExpiresAt`.
5. Confirm the completion meter remains `Draft` and below 100%.
6. Reload the page and confirm the attempted completion did not mark the record complete.

Expected result: The API rejects completion while required profile fields are missing, and the stored profile remains a draft.

Reason: This distinguishes server-enforced completion requirements from a visual progress indicator. A button click alone must not convert incomplete data into a completed profile.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-P03: Complete The Synthetic Company Profile

Category: Company profile.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: Profile.

Steps:

1. Stay on Profile.
2. Enter the Profile values from the `Test Data` table:
 - Legal entity: Blue Ridge Federal Support LLC.
 - DBA: Blue Ridge Support.
 - UEI: UAT123ABC456.
 - CAGE: 7UAT1.
 - SAM expires: 2027-07-31.
 - Role: Subcontractor.
 - Agency customers: DHS synthetic UAT customer.
 - Products and services: Synthetic non-CUI help desk and compliance support services..
 - Employees: Small.
 - Revenue: Small.
3. In `NAICS codes`, keep one row and select its `Primary` radio button.
4. Enter 541512 for `Code` and `Computer Systems Design Services` for `Title`.
5. Enter UAT synthetic value – not an SBA determination for `Size basis`.
6. Choose `Small` for the synthetic `Status`. Do not use this selection as an actual SBA size determination.
7. Enter the location values:
 - Location: UAT Headquarters.
 - Street: 100 Test Plaza.
 - City: Arlington.
 - State: VA.
 - Postal code: 22201.
 - Country: USA.
8. Enter Synthetic Microsoft 365 and managed endpoint environment used only for No-CUI UAT. in `IT summary`.
9. Choose `FCI` only for `FCI/CUI posture`.

10. Enter Microsoft 365, synthetic help desk in Key systems.
11. Leave Uses external service provider unchecked unless the UAT scenario specifically requires one.
12. Click Complete profile.
13. Confirm the status message is Profile complete..
14. Confirm the completion meter shows Complete and 100%.
15. Reload the page and confirm the completed values persist.

Expected result: The synthetic company profile is saved as complete and displays 100% after all server-required fields are present.

Reason: Company profile facts provide tenant business context for applicability, contracts, renewals, readiness work, and reports. The synthetic size-status values are test inputs, not legal or SBA determinations.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-P04: Verify Profile Read-Only Access And Tenant Isolation

Category: Company profile.

Roles: Contributor, Auditor, Advisor, and Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tab: Profile.

Steps:

1. Switch to the Contributor persona and apply the context.
2. Click Profile and confirm the completed synthetic profile is visible.
3. Confirm the profile fields and Save draft and Complete profile actions are disabled.
4. Repeat steps 1 through 3 as Auditor and Advisor.
5. Attempt a direct PUT /api/company-profile as each read-only profile role and confirm the response is 403.
6. Reload the profile and confirm none of the denied requests changed any value or created a profile update audit event.
7. Using an authorized test harness, request GET /api/company-profile under a different tenant context that has no profile.
8. Confirm the response is 204 with no profile body; it must not return Blue Ridge Federal Support LLC or its identifiers.
9. Switch back to the Compliance Manager persona before continuing.

Expected result: Contributor, Auditor, and Advisor can view the selected tenant's profile but cannot modify it. A different tenant receives only its own profile or an empty response.

Reason: The UI must reflect server-authoritative permissions, but the acceptance boundary is the API denial and unchanged persisted state. Tenant isolation prevents business identity and registration metadata from leaking across workspaces.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-03: Create The No-CUI Contract Record

Category: One configured No-CUI readiness workflow that shows contract metadata.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Contracts`.

Steps:

1. Click the `Contracts` tab.
2. Click `New contract` if an existing contract is selected.
3. In `Create contract record`, enter the contract data from the `Test Data` section.
4. Confirm `FCI/CUI posture` is `FCI only`.
5. Click `Create contract`.
6. Select `DEMO-NC-26-0007` in `Contract records`.

Expected result: The selected contract displays its number, title, agency or prime, relationship, type, status, dates, posture, place of performance, and description.

Reason: Contract metadata is the anchor for clause attachment, obligation generation, evidence scope, reporting scope, and audit history.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-04: Upload Contract Document Metadata

Category: One configured No-CUI readiness workflow that shows contract metadata.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Contracts`.

Steps:

1. Stay on `Contracts`.
2. Select `DEMO-NC-26-0007`.
3. In `Required before contract` or `evidence work`, confirm the current No-CUI notice is acknowledged. If it is not, check all four `Required user acknowledgement statements` and click `I acknowledge the No-CUI upload limitation`.
4. Confirm the acknowledgement status is `Acknowledged` before continuing.
5. In `Documents`, set `Document type` to `Contract`.
6. Set `Contract document classification` to `FCI`.
7. Choose a local text file named `demo-nc-contract.txt` containing the allowed synthetic text from `Test Data`.
8. Check `I confirm this file does not contain CUI, classified information, export-controlled data, ITAR data, or sensitive government-furnished information`.
9. Click `Upload document`.
10. Confirm the document appears in the document list with `FCI classification`, `accepted validation`, and `clean malware status`.
11. Click `Start extraction`.
12. Confirm the status progresses from `Queued` or `Processing` to `Completed` and that the candidate count is displayed.

Expected result: The app accepts the synthetic FCI-only text document, stores it privately after malware scanning, and completes tenant-scoped clause extraction.

Reason: This proves the workflow can process an allowed synthetic contract document without accepting prohibited CUI content or treating extracted candidates as reviewed clauses.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Contract Deliverables UAT

Current-state label: Implemented.

Implementation evidence: Deliverables are displayed inside the selected contract on the `Contracts` tab. Tenant-scoped list, create, and update endpoints require `ViewContracts` or `ManageContracts`. Focused tests cover contract-detail display, calendar-task creation, overdue calculation, and deliverable audit events.

UAT-D01: Create A Contract Deliverable

Category: Contract deliverables.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Contracts`.

Prerequisite: UAT-03 is complete and contract `DEMO-NC-26-0007` is selected.

Steps:

1. Click the `Contracts` tab.
2. Select contract `DEMO-NC-26-0007` from `Contract records`.
3. Scroll within the contract detail until the `Deliverables` section is visible. This section is below `Attached clauses` and above `Documents` in the current UI.
4. Enter `Monthly service status report - synthetic` in `Name`.
5. Enter `Contracts` in `Owner`.
6. Enter `2026-08-31` in `Due date`.
7. Choose `Not started` in `Deliverable status`.
8. Enter `Synthetic monthly performance summary containing no customer CUI or sensitive government data.` in `Deliverable description`.
9. Click `Add deliverable`.
10. Confirm the message is `Deliverable added to the contract calendar..`
11. Confirm the deliverable list shows its name, owner, due date, description, and `Not started` status.
12. Reload the page, reselect `DEMO-NC-26-0007`, return to `Deliverables`, and confirm the record persists.

Expected result: The deliverable is created under the selected contract and remains visible after reload.

Reason: A contract deliverable must retain its contract association, owner function, due date, description, and lifecycle state. Persistence after reload distinguishes a saved record from temporary UI state.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-D02: Verify Deliverable Calendar Linkage

Category: Contract deliverables.

Role: Compliance Manager or Contributor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Calendar`.

Steps:

1. Click the `Calendar` tab.
2. Set the calendar date range so that it includes `2026-08-31`.
3. Apply a source or module filter for contract deliverables if that filter is available; otherwise review the complete date range.
4. Find `Monthly service status report - synthetic`.
5. Confirm its date is `2026-08-31` and its owner is `Contracts` when displayed.
6. Return to `Contracts`, select `DEMO-NC-26-0007`, and confirm the source deliverable still shows the same due date and owner.

Expected result: The dated deliverable appears in the tenant-scoped calendar without requiring duplicate manual task entry.

Reason: Calendar linkage turns contract performance dates into visible operational work. The API creates or synchronizes a calendar task when the deliverable is saved.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-D03: Verify Overdue State And Status Update

Category: Contract deliverables.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Contracts`, then `Calendar`.

Steps:

1. On `Contracts`, select `DEMO-NC-26-0007` and return to `Deliverables`.
2. Create a second deliverable using the negative-test values from `Test Data`:
 - Name: `Overdue corrective-action summary - synthetic`.
 - Owner: `Compliance`.
 - Due date: `2026-07-15`.
 - Deliverable status: `In progress`.
 - Deliverable description: `Synthetic past-due record used to verify overdue presentation`.
3. Click `Add deliverable`.
4. Confirm the new row displays both `In progress` and `Overdue`.
5. In the status selector for `Monthly service status report - synthetic`, choose `Submitted`.
6. Confirm the message is `Deliverable updated`. and the row displays `Submitted`.
7. Reload the page and confirm `Submitted` persists.
8. Click `Calendar`, include both deliverable dates in the date range, and confirm both records are visible.

Expected result: A past-due incomplete deliverable is flagged overdue, and an authorized status update persists and synchronizes with the calendar task.

Reason: Overdue state is derived from due date and incomplete status, while lifecycle status is an explicit user-controlled value. Testing both catches date-calculation and persistence failures.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-D04: Verify Deliverable Read-Only Access And Tenant Isolation

Category: Contract deliverables.

Roles: Contributor, Auditor, Advisor, and Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it Passed.

Tab: `Contracts`.

Steps:

1. Switch to the `Contributor` persona and apply the context.
2. Click `Contracts`, select `DEMO-NC-26-0007`, and scroll to `Deliverables`.
3. Confirm existing deliverables are visible but the create form and status selectors are disabled.
4. Repeat steps 1 through 3 as `Auditor`.
5. Attempt direct `POST /api/contracts/{contractId}/deliverables` and `PUT /api/contracts/{contractId}/deliverables/{deliverableId}` requests as `Contributor` and `Auditor`.
6. Confirm each mutation returns 403, existing deliverable values remain unchanged, no additional calendar task is created, and no deliverable audit event is written.
7. Switch to `Advisor` and confirm the create form and status selector are enabled because `Advisor` currently has `ManageContracts`.
8. Do not create another record as `Advisor` unless that role behavior is specifically under test.
9. Using an authorized test harness with a different tenant context, request `GET /api/contracts/{contractId}/deliverables` using the original tenant's contract ID.
10. Confirm the response is 404 and does not disclose deliverable names, dates, owners, or IDs.
11. Switch back to the `Compliance Manager` persona before continuing.

Expected result: `Contributor` and `Auditor` can view but cannot mutate deliverables. `Advisor` can manage deliverables under the current permission catalog. Cross-tenant access returns 404 without record disclosure.

Reason: Deliverables inherit contract authorization. Testing UI state alone is insufficient; direct API denial, unchanged tasks and audit history, and cross-tenant non-disclosure prove the boundary.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-05: Search Source-Backed Clauses

Category: Attached or reviewed clauses.

Role: `Compliance Manager` or `Advisor`.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it Passed.

Tab: Obligations.

Steps:

1. Click the Obligations tab.
2. Find Clause library search.
3. Search 52.204-21.
4. Record the published clause ID for the matching FAR clause.
5. Repeat for 52.204-25.
6. Repeat for 52.204-27.
7. Confirm each result includes clause number, title, source URL, confidence or review state when displayed.

Expected result: Published clause records can be located before attachment.

Reason: Clause-driven obligations should come from reviewed source-backed content, not free-form user text.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-06: Attach Clauses To Contract

Category: Attached or reviewed clauses.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tab: Contracts.

Steps:

1. Click the Contracts tab.
2. Select DEMO-NC-26-0007.
3. In Attached clauses, paste the published clause ID for 52.204-21.
4. Enter Manual UAT tagging from synthetic contract text. for Attachment reason.
5. Enter demo-nc-contract.txt for Source document reference.
6. Click Attach clause.
7. Repeat steps 3 through 6 for 52.204-25 and 52.204-27.
8. Confirm attached rows show clause number, title, source URL, and review metadata when available.

Expected result: Three clauses are attached to the contract.

Reason: Attachment creates the bridge from contract metadata to source-backed obligation generation.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-07: Generate And Review Contract Obligations

Category: Generated obligations.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tab: Contracts, then Obligations.

Steps:

1. Stay on Contracts.
2. In the attached clause row for 52.204-21, use the available generate action if visible.
3. If generation occurs automatically after attachment, proceed to the next step.
4. Click the Obligations tab.
5. In Obligation work queue, filter by contract DEMO-NC-26-0007.
6. Apply any useful filters: Risk = High, Owner = Security or IT/security, Module = Cybersecurity, Source = 52.204-21.
7. Open the matching obligation with View details.
8. Confirm detail sections include Why it applies, Required action, Owner, Source, Confidence, Last reviewed, Evidence examples, and Flow-down.

Expected result: At least one source-backed obligation appears for the contract, preserving clause and source metadata.

Reason: The acceptance point is not merely that a clause is attached. The system should turn reviewed clause mappings into actionable work while preserving provenance.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-08: Update Obligation Status

Category: Owner/status tracking.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tab: Obligations.

Steps:

1. Stay in the opened obligation detail.
2. In Update status, choose In progress.
3. Click Save status.
4. Confirm the status shown in detail or the work queue changes to In progress.

Expected result: The obligation status updates for the selected tenant-scoped contract obligation.

Reason: Status tracking is the operational control that turns static compliance content into an active readiness workflow.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-09: Assign Obligation Owner

Category: Owner/status tracking.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Obligations`.

Prerequisite: Priya Shah and Devin Brooks are active members of the selected tenant. Confirm both names appear under `Switch user` before assigning the obligation. If the selector is disabled or either name is absent, complete the tenant invitation/activation workflow first.

Steps:

1. Stay in the opened obligation detail.
2. In `Assign by`, choose `Tenant member`.
3. In `Tenant member`, choose `Devin Brooks`.
4. Leave `Also send assignment email` checked.
5. Click `Assign owner`.
6. Reload, reopen the obligation detail, and confirm `Currently assigned to`, `Assign by`, and `Tenant member` show `Devin Brooks`.
7. In the local test context, use `Switch user` to select `Devin Brooks` and apply the context.
8. Confirm the notification bell shows an unread direct assignment. Open it, then return to `Obligations` and select `My assignments`.
9. Switch the development user back to `Priya Shah`, assign the same obligation by `Role`, and choose `Compliance manager`.
10. Reload, reopen the detail, and confirm the saved role remains displayed.
11. Select `Role assignments` and confirm the obligation appears with the role-queue count.
12. Switch to an active `Compliance Manager` persona and confirm the bell contains the role-assignment notification.

Expected result: A tenant-member or role assignment remains visible after reload in both the persistent assignment summary and assignment controls. A directly assigned member receives an in-app notification and can find the obligation under `My assignments`. Active members of an assigned role receive one deduplicated in-app notification and can find the obligation under `Role assignments`. When direct-assignment email delivery is configured and the member's `Assignment emails` preference is enabled, an email is queued asynchronously. Role-assignment email remains disabled.

Reason: Direct and role ownership must survive reload, provide an explicit queue, and make eligible recipients aware of new work. Role notification fan-out remains tenant-scoped and in-app only to avoid ungoverned mass email. Asynchronous direct-assignment email delivery prevents an external email-provider failure from rolling back the obligation assignment.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-10: Acknowledge No-CUI Evidence Rules

Category: Allowed evidence metadata.

Role: Contributor or Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Evidence`.

Steps:

1. Click the Evidence tab.
2. Find No-CUI acknowledgement.
3. Read the notice and confirm it prohibits real customer CUI in No-CUI mode.
4. Click I acknowledge the No-CUI upload limitation.
5. Confirm the status changes to Acknowledged.

Expected result: Evidence controls become available only after acknowledgement.

Reason: The acknowledgement is an operational safety gate. It educates users before they create or upload evidence in a No-CUI tenant.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-11: Create Allowed Evidence Metadata

Category: Allowed evidence metadata.

Role: Contributor or Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tab: Evidence.

Steps:

1. Stay on Evidence.
2. Find Evidence metadata.
3. Click New evidence.
4. Enter the evidence data from the Test Data section.
5. For Obligations, paste the generated FAR 52.204-21 obligation ID if visible; otherwise leave blank and rely on the Controls link.
6. For Controls, enter AC.L1-3.1.1.
7. Click Create metadata.
8. Confirm the record appears in Evidence list.

Expected result: The evidence record appears with title, type, owner, status, dates, tags, classification, and control or obligation links.

Reason: Evidence metadata lets the system track proof without requiring storage of sensitive file content.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-12: Negative Evidence Classification Check

Category: Allowed evidence metadata.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: Evidence.

Steps:

1. Stay on Evidence.
2. Select the synthetic evidence item or create a new test evidence record.
3. Attempt to classify it as CUI while the tenant remains in NoCui mode.
4. Enter `Negative UAT check: NoCui mode should not accept CUI evidence.` as the classification reason.
5. Submit the classification change.

Expected result: The workflow is rejected or blocked by the No-CUI policy.

Reason: A positive-only UAT misses the main safety guarantee. This negative test proves that No-CUI mode does not silently accept CUI-labeled evidence.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

CMMC Module UAT

Current-state label: Implemented.

Implementation evidence: The CMMC tab exposes CMMC and NIST workspace, Readiness assessments, Control readiness, and POA&M remediation. The API exposes tenant-scoped CMMC assessment, control, POA&M, affirmation, and readiness report routes under `/api/cmmc/...` and `/api/reports/cmmc-readiness`. Focused tests cover Level 1 and Level 2 assessment creation, control baseline loading, control status updates, evidence/task/asset/POA&M links, traceability validation, POA&M calendar linkage, affirmation calendar/reminder behavior, CMMC readiness report language, RBAC, and tenant isolation.

Important posture limit: This module tracks CMMC readiness work only. It does not certify CMMC compliance, provide an assessor determination, authorize real CUI storage, or replace qualified CMMC/security review.

UAT-C01: Create A No-CUI CMMC Readiness Assessment

Category: CMMC readiness module.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: CMMC.

Steps:

1. Click the CMMC tab under the Assurance navigation group.
2. Confirm the page heading is CMMC and NIST workspace.
3. Confirm the MVP posture banner says the current tenant is NoCui and that real CUI, classified, and export-controlled data are blocked.
4. In the assessment form, enter the CMMC values from the `Test Data` section:
 - Assessment name: No-CUI Level 1 readiness workspace.
 - Target level: Level 1.
 - Framework: FAR basic safeguarding.
 - Status: In progress.
 - Started: 2026-06-15.
 - Affirmation due: 2027-06-15.
 - Owner: Security.
 - Contract link: DEMO-NC-26-0007, if that contract appears in the selector; otherwise leave `No contract` selected.

5. Click `Create assessment`.
6. Confirm the assessment appears under `Readiness assessments`.
7. Confirm the assessment card shows:
 - `Level 1`.
 - `In Progress`.
 - `Owner: Security`.
 - `Affirmation due: 2027-06-15`.
 - `Complete: a percentage value`.
 - `Implemented: implemented count over total control count`.
 - `Open POA&M: a count`.

Expected result: A Level 1 CMMC readiness assessment is created and displayed with owner, dates, progress summary, and POA&M summary.

Reason: This verifies the CMMC module is part of the No-CUI readiness workflow without claiming CMMC certification or CUI-ready operation.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-C02: Review The CMMC Control Readiness Baseline

Category: CMMC readiness module.

Role: Compliance Manager or Auditor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: CMMC.

Prerequisite: UAT-C01 is complete and at least one CMMC readiness assessment is listed.

Steps:

1. Stay on the CMMC tab.
2. Find `Control readiness`.
3. Locate control `AC.L1-3.1.1` or another visible Level 1 control.
4. Confirm each visible control card includes:
 - Control ID and title.
 - Status, such as `Not Started`, `Implemented`, `Partially Implemented`, `Not Applicable`, or `Needs Review`.
 - Assessment result, such as `Not Assessed`, `Met`, or `Not Met`.
 - Source confidence.
 - Family.
 - Source.
 - Reviewed.
 - Evidence.
 - Tasks.
 - Assets.
 - Open POA&M.
5. If a control has no linked evidence, confirm a `Data quality` warning appears.
6. API-harness-only step: if an authorized API test harness is available, retrieve the `assessmentId` from `GET /api/cmmc/assessments`. The assessment ID is not currently displayed in the UI.
7. API-harness-only step: call `GET /api/me/access` and confirm the test actor has `ManageCmmc` before attempting a control-status update.
8. API-harness-only step: update a control status through `PATCH /api/cmmc/assessments/{assessmentId}/controls/{controlId}` and confirm the assessment completion percentage changes after reloading the UI or calling `GET /api/cmmc/assessments/{assessmentId}`.

9. API-harness-only step: attempt to mark a control Implemented with Met result but no linked reviewed evidence through the API.
10. Confirm the API rejects the request with a validation error referencing linked evidence.
11. API-harness-only step: attempt to link evidence from another tenant through the same control-status endpoint.
12. Confirm the API rejects the cross-tenant evidence link and does not change the control.

Postman/local development note: local API calls must send the same tenant and development-auth context as the app, including X-Gccs-Dev-Auth, X-Gccs-Tenant, X-Gccs-Dev-Tenant, X-Gccs-Dev-User, X-Gccs-Dev-Email, and X-Gccs-Dev-Role. A 403 means the caller is authenticated but not authorized for the requested tenant or permission.

Expected result: The control baseline is visible in the UI, and the API enforces traceability checks for implemented controls and cross-tenant evidence links.

Reason: CMMC readiness is not just a checklist label. Control status must stay tied to source metadata, evidence traceability, and tenant isolation.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-C03: Create A CMMC POA&M Remediation Item

Category: CMMC readiness module.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tab: CMMC, then Calendar.

Prerequisite: UAT-C01 is complete and Control readiness has loaded controls.

Steps:

1. Stay on the CMMC tab.
2. Find POA&M remediation.
3. In the POA&M form, enter:
 - Control: AC.L1-3.1.1.
 - Risk: High.
 - Status: Open.
 - Owner: Security.
 - Due date: 2026-07-15.
 - Gap: Synthetic UAT gap: document annual access review evidence.
 - Remediation plan: Upload synthetic access review summary and link it to the control.
4. Click Create POA&M.
5. Confirm the POA&M item appears under POA&M remediation.
6. Confirm the item shows:
 - Control AC.L1-3.1.1.
 - Risk High.
 - Status Open.
 - Owner Security.
 - Due date 2026-07-15.
 - Task: Linked, when displayed.
7. Click the Calendar tab.
8. Set the calendar date range so it includes 2026-07-15.
9. Confirm a CMMC/POA&M-related calendar task appears for the remediation item.
10. If other calendar items appear for contracts or deliverables, ignore them for this CMMC test.
11. Confirm the relevant calendar item shows Module: CMMC.

Expected result: The POA&M item is created, linked to the selected control, and appears as CMMC remediation work on the calendar.

Reason: A readiness workspace must create owner-tracked remediation work, not just display static gaps.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-C04: Generate A CMMC Readiness Report

Category: CMMC readiness module.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Reports`.

Prerequisite: UAT-C01 is complete. UAT-C03 is recommended so the report has POA&M content.

Steps:

1. Click the `Reports` tab.
2. Find CMMC readiness.
3. In `Assessment`, choose `No-CUI Level 1 readiness workspace` or the CMMC assessment created in UAT-C01.
4. Click `Generate readiness`.
5. Confirm a generated report appears under `Recent generated reports`.
6. Click the generated CMMC readiness report card to open the report detail panel.
7. Confirm the report detail metric cards include CMMC readiness values:
 - `Assessment`.
 - `Target level`.
 - `Control rows`.
 - `Open gaps`.
 - `Open POA&M`.
 - `Evidence links`.
8. Confirm the section below the metric cards is named `Report content`. Do not expect a literal `Readiness summary` heading in the current UI.
9. Confirm the report wording says it is workflow guidance only and does not claim legal advice, certification decision, assessor determination, contracting-officer determination, government approval, or government endorsement.
10. API-harness-only step: if an authorized API test harness is available, generate the same report through `POST /api/reports/cmmc-readiness?assessmentId={assessmentId}`. Retrieve `assessmentId` from `GET /api/cmmc/assessments`; it is not currently displayed in the UI.
11. Confirm the API creates a tenant-scoped report for an authorized report manager and returns 403 for a user without report-generation permission.
12. Confirm the API rejects a report that would include `Unknown`, `Prohibited`, or real CUI evidence in a No-CUI tenant.

Expected result: A CMMC readiness report is generated as a tenant-scoped workflow artifact and preserves No-CUI/report-language boundaries.

Reason: The CMMC module is incomplete without a report handoff. The report must communicate readiness status without over-claiming certification, legal, assessor, contracting-officer, or government determinations.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-13: Generate Current Report Artifact

Category: A current report artifact.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Reports`.

Steps:

1. Click the `Reports` tab.
2. In `Compliance status`, click `Generate status`.
3. Confirm a report appears under `Recent generated reports`, then click its card and confirm the report detail panel opens. Reload the page and confirm the report remains listed.
4. In `Evidence package builder`, enter `Prime review evidence package - No-CUI UAT` for `Package title`.
5. Select the generated obligation if available.
6. Select contract `DEMO-NC-26-0007`.
7. Select control `AC.L1-3.1.1` if available.
8. Leave `Include draft/rejected evidence when authorized` unchecked.
9. Click `Generate package`.
10. Confirm the package appears under `Recent generated reports` or `Approved evidence packages`, then click its card and confirm the package detail panel opens.
11. In the opened detail panel, confirm the report wording says it is workflow guidance and does not claim legal advice, certification decision, assessor determination, contracting-officer determination, or government endorsement.

Expected result: A current report artifact is generated from tenant-scoped obligations, contract data, and approved evidence metadata.

Reason: Reports are the handoff artifact for internal reviews and prime-review preparation. The acceptance test also verifies that the report avoids certification and legal overclaims.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-14: Verify Audit History For Created Records

Category: Audit history.

Role: Owner, Admin, or Advisor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Settings`.

Steps:

1. Click the `Settings` tab.
2. Find `Audit log`.
3. Set `Action` to `Created`.
4. Set `Entity` to `CompanyProfile`.
5. Click `Filter`.
6. Confirm a created event exists for `Blue Ridge Federal Support LLC`; if the profile existed before this UAT, change `Action` to `Updated` and confirm an updated event instead.

7. Set Action back to Created, change Entity to Contract, and click Filter.
8. Confirm a created event exists for DEMO-NC-26-0007 or the created contract ID.
9. Change Entity to ContractDeliverable.
10. Click Filter.
11. Confirm created events exist for Monthly service status report - synthetic and Overdue corrective-action summary - synthetic.
12. Set Action to Updated and click Filter.
13. Confirm an updated event records the Submitted status for Monthly service status report - synthetic.
14. Set Action back to Created, change Entity to ContractClause, and click Filter.
15. Confirm created events exist for attached clauses.
16. Change Entity to EvidenceItem.
17. Click Filter.
18. Confirm a created event exists for MFA configuration summary - synthetic.
19. Change Entity to CmmcAssessment.
20. Click Filter.
21. Confirm a created event exists for No-CUI Level 1 readiness workspace.
22. Change Entity to CmmcPoamItem.
23. Click Filter.
24. Confirm a created event exists for the synthetic CMMC POA&M item.
25. Change Entity to Report.
26. Click Filter.
27. Confirm created events exist for the generated compliance status, evidence package, and CMMC readiness reports.

Expected result: The audit log shows tenant-scoped history for the company profile, contract, deliverable, clause, evidence, CMMC assessment, CMMC POA&M, and report actions.

Reason: Audit history is the accountability trail. It should show who did what, when, and to which tenant-scoped entity.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-15: Verify Audit Access By Role

Category: Audit history.

Role: Auditor, Contributor, Owner.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tab: Settings.

Steps:

1. As Owner, confirm the Settings tab and Audit log are visible.
2. As Contributor, attempt to access Settings or audit log review.
3. Confirm Contributor cannot view audit logs.
4. As Auditor, attempt to access audit logs.
5. Confirm Auditor cannot access audit logs unless the implementation grants ViewAuditLog.
6. As Advisor, confirm audit visibility is available if assigned the Advisor role.

Expected result: Audit log access follows the role permission matrix.

Reason: Audit logs can expose sensitive operational metadata. Access should be restricted to roles with explicit audit permission.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Public Demo And Platform Follow-Up UAT

Current-state label: Implemented for public request intake, request-detail capture, platform review, responses, appointments, and follow-up records. Email and HubSpot delivery are environment-dependent.

Implementation evidence: The public UI exposes demo request and detail pages. Platform pages expose demo-request list, calendar, response, follow-up preview, and appointment confirmation. API and frontend tests cover validation, authorization, provider failure, and user-visible states.

UAT-M01: Submit A Synthetic Public Demo Request

Category: Public demo request intake.

Role: Anonymous visitor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Page: Public landing page and demo-request detail page.

Steps:

1. Open the public FeDril landing page in a clean browser profile.
2. Confirm the page identifies FeDril as a No-CUI compliance-management product and does not claim certification, government approval, legal advice, or secure CUI storage.
3. Open the demo request form and enter the Public demo values from Test Data.
4. Submit once, refresh, and avoid resubmitting until the result is known.
5. Confirm a success state appears without exposing an internal tenant ID, provider response, stack trace, or token.
6. Open the single-use detail link from the synthetic mailbox when delivery is configured; otherwise mark only the delivery-dependent step `Blocked by environment`.
7. Enter synthetic context and submit it once.
8. Repeat with malformed email, oversized text, script markup, duplicate submission, and an expired or changed token.

Expected result: Valid synthetic data creates one bounded request and detail record. Invalid, duplicate, expired, or manipulated input is rejected safely without script execution or internal-data disclosure.

Reason: Public unauthenticated input must be usable without creating an injection, spam, token-disclosure, or sensitive-data intake path.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-M02: Review And Respond To A Demo Request

Category: Platform demo operations.

Role: Platform Operator with the exact demo-management permission.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Navigation: Platform operations -> Demo requests and calendar.

Prerequisite: UAT-M01 created the synthetic request.

Steps:

1. Open the platform demo-request list and locate `Blue Ridge Demo Company - Synthetic`.
2. Open the request and confirm only the submitted synthetic fields are displayed.
3. Record a synthetic response and schedule a synthetic appointment.
4. Open the platform calendar and confirm the appointment appears once with the correct time zone.
5. Generate the development follow-up preview only where that development-only route is enabled.
6. Retry the response and appointment request, then simulate an unavailable email or CRM provider in an approved test environment.
7. Sign in as a normal Tenant Owner and call the platform endpoints directly.

Expected result: Authorized platform actions persist and remain traceable; duplicate/provider-failure behavior is explicit and bounded. A tenant role receives 403, and development preview behavior is unavailable outside Development.

Reason: Public intake and internal follow-up cross an external-input and platform-privilege boundary that must remain separate from tenant administration.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-M03: Verify Platform Customer List And Detail Administration

Category: Platform customer administration.

Role: Platform Operator with customer-read/subscription permissions; normal Tenant Owner for denial.

Tenant context: Platform scope, using only the synthetic Pilot and Paid tenants created in UAT-T01 through UAT-T03.

Test data: Use the exact synthetic fixtures in `Test Data` and the recorded immutable Tenant IDs.

Preconditions: The synthetic tenants exist. Do not inspect or modify real customer records.

Navigation: Platform operations -> Customers -> synthetic customer detail.

Steps:

1. Open the platform customer list and locate both synthetic tenants by customer reference.
2. Filter/page the list and confirm counts, status, subscription type, and No-CUI mode match platform APIs.
3. Open each detail page and reconcile tenant, Owner, onboarding, invitation, subscription, and lifecycle history.
4. Perform an allowed synthetic subscription transition and confirm the detail refreshes once.
5. Try a random/Tenant B identifier without the required platform permission and inspect the error for metadata leakage.
6. Sign in as a normal Tenant Owner and call the customer list/detail APIs directly.

Expected result: Platform customer data is visible only to authorized platform operators, reconciles with onboarding/subscription state, and does not leak through filtering, pagination, identifiers, or errors.

Reason: Central customer administration aggregates every tenant and therefore has a larger disclosure radius than a tenant-scoped screen.

Security expectation: Platform permissions remain distinct from tenant roles. Denied requests must not disclose customer names, tenant IDs, subscription references, owner identities, or counts.

Evidence to capture: Platform permission response, sanitized list/detail API and UI results, pagination/filter evidence, allowed lifecycle audit, denials, and no-side-effect proof.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Identity, Membership, Workspace, And Subscription UAT

Current-state label: Implemented. Production login, MFA, token expiration, and logout require a production-like identity provider; local development headers are not identity proof.

Implementation evidence: The API exposes tenant invitations, membership lifecycle, `/api/me/access`, tenant selection, and platform subscription lifecycle endpoints. Focused tests cover invitations, membership status, workspace selection, role claims, authentication boundaries, and subscription transitions.

UAT-I01: Invite, Accept, Resend, Revoke, And Expire A Tenant User

Category: Tenant user administration.

Roles: Admin, invited Contributor, and unauthorized Auditor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Settings`; invitation activation page.

Steps:

1. As Admin in Tenant A, invite `new.contributor+uat@example.com` as Contributor.
2. Confirm the pending invitation appears once and an audit event records the safe invitation metadata.
3. Resend only after the configured delivery state permits it; record `Blocked by environment` if no provider exists.
4. Attempt acceptance with a different synthetic identity, then accept with the exact invited identity.
5. Confirm one active Tenant A membership exists and the invitation is terminal.
6. Replay the link and confirm it cannot create another membership.
7. Create a second invitation, revoke it, and confirm it cannot be accepted; create a third, expire it, and confirm the same.
8. As Auditor, attempt invitation and membership mutations through the API.

Expected result: Only authorized administrators create or manage invitations. Exact-identity acceptance creates one membership; replayed, revoked, expired, wrong-identity, and unauthorized requests create no membership or token-bearing log entry.

Reason: Invitation lifecycle is an account-provisioning boundary and must resist identity mismatch, replay, role escalation, and duplicate membership creation.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-I02: Deactivate A Member And Verify Access Removal

Category: Membership lifecycle and session revocation.

Roles: Admin and Contributor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tenant: Tenant A.

Steps:

1. Sign in as the active Contributor and capture `GET /api/me/access` and one permitted tenant read.
2. In a separate Admin session, deactivate the Contributor membership.
3. Retry the tenant read and a permitted mutation from the original Contributor session.

4. Reload and use browser back on a protected route.
5. Reactivate only if the implemented lifecycle permits it, then capture the restored permission state.
6. Attempt to deactivate a Tenant B membership while operating in Tenant A.

Expected result: Deactivation removes access at the documented authorization boundary even for an existing session. Cross-tenant membership identifiers are not disclosed, and unauthorized operations create no business side effects.

Reason: Hiding a user in Settings is insufficient; the server must stop active access when membership is no longer active.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-I03: Switch Tenants And Exercise Subscription Lifecycle

Category: Tenant selection and commercial lifecycle metadata.

Roles: Multi-tenant Advisor and Platform Operator.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tenants: Tenant A, Tenant B, and disposable pilot subscription fixtures.

Steps:

1. As an Advisor with explicit memberships in A and B, list available tenants and select A.
2. Open a known A record, switch to B, reload, and confirm no A data remains in views, filters, cached details, or exports.
3. Logout and sign in again; verify the documented workspace selection behavior.
4. Attempt to select a random, pending, archived, or unassigned tenant ID.
5. As Platform Operator, extend a pilot, convert a pilot to paid, place a disposable subscription in grace/expired/cancelled states, and retry a stale transition.
6. Confirm subscription actions never change the tenant data-handling mode or grant platform permissions to a tenant user.

Expected result: Selection is limited to active memberships and clears prior-tenant data. Subscription transitions are authorized, version-safe, audited, and remain commercial metadata rather than CUI authorization.

Reason: Tenant switching and subscription state are common sources of cached cross-tenant data and accidental privilege coupling.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Dashboard UAT

Current-state label: Implemented for tenant-scoped overview cards, alerts, and drill-down navigation.

UAT-DB01: Reconcile Executive Dashboard Metrics And Alerts

Category: Executive readiness dashboard.

Role: Compliance Manager and Auditor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Dashboard`.

Prerequisite: Original contract, obligation, evidence, deliverable, CMMC, and POA&M fixtures exist.

Steps:

1. Open Dashboard and wait for loading to finish.
2. Record control coverage, contract risk, overdue, high-risk, evidence/document review, and No-CUI notice metrics.
3. Compare each count with the corresponding filtered module list and API response.
4. Open each available alert or drill-down and confirm it resolves to the same tenant-scoped record.
5. Switch to Tenant B and verify A names, counts, and alert identifiers disappear.
6. Simulate an API error and confirm the dashboard shows an explicit error rather than cached or invented readiness values.

Expected result: Dashboard metrics and alerts reconcile with current module data, respect tenant and role boundaries, and represent loading, empty, and failure states accurately.

Reason: An executive summary that is stale or cross-tenant can materially mislead readiness decisions even when underlying records are correct.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Contract Intelligence And Applicability UAT

Current-state label: Implemented for document extraction candidates, clause review actions, applicability facts/rules, suggested obligations, expert review, contract size checks, and optional entity lookup. Provider and worker paths are environment-dependent.

UAT-X01: Review Extracted Clause Candidates

Category: Contract document extraction and human review.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Contracts`.

Prerequisite: UAT-04 completed extraction for `demo-nc-contract.txt`.

Steps:

1. Open the document extraction results and confirm each candidate identifies the source document and extracted clause reference.
2. Edit one candidate using harmless synthetic text.
3. Mark separate candidates `Accepted`, `Rejected`, and `Needs clarification` with reasons.
4. Supersede one prior decision with a replacement candidate where supported.
5. Reload and confirm review state, reviewer, date, reason, and history persist.
6. Repeat a stale or duplicate decision and attempt a Tenant B candidate ID.
7. Confirm accepted extraction does not automatically publish unreviewed compliance content or accept a prohibited classification.

Expected result: Candidate decisions are human-controlled, tenant-scoped, conflict-safe, source-linked, and audited; stale, duplicate, and cross-tenant actions do not corrupt history.

Reason: Extracted text is a draft input. It must not silently become authoritative compliance guidance.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-X02: Evaluate Applicability And Suggested Obligations

Category: Applicability facts, rules, suggested obligations, and expert review.

Role: Compliance Manager; qualified reviewer for final review actions.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it Passed.

Tabs: Profile, Contracts, and Obligations; API harness where no dedicated UI exists.

Steps:

1. Save synthetic applicability facts derived from the completed profile and contract.
2. Evaluate an available reviewed rule and record its source, effective date, confidence, and rationale.
3. Create a suggested obligation from governed source identifiers, not free-form tenant or source claims.
4. Edit the draft, then approve one, reject one, and escalate one to the expert-review queue.
5. Resolve the expert-review item with a synthetic rationale.
6. Repeat approval and submit stale, unsupported-source, draft-content, and Tenant B references.
7. Confirm only eligible reviewed content becomes customer-visible or generates downstream work.

Expected result: Applicability and suggestions preserve source provenance and explicit human review. Unsupported, stale, duplicate, or cross-tenant input fails without publishing or generating work.

Reason: Applicability output can be mistaken for legal advice unless sources, state, rationale, and reviewer accountability remain visible.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-X03: Verify SAM Lookup And Contract Size Checks

Category: Optional entity lookup and size-assistance workflows.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it Passed.

Tabs: Profile and Contracts.

Steps:

1. Search the configured SAM provider using the synthetic UEI; if the provider is disabled, record the lookup step `Blocked by environment`.
2. Review returned provenance and apply only the intended synthetic fields.
3. Confirm the user must explicitly save the profile after applying a lookup.
4. Create a contract size check using the synthetic NAICS code and clearly labeled test inputs.
5. Verify the result includes source and limitation language and is not called an SBA determination.
6. Submit invalid NAICS, missing source data, a Tenant B contract, and concurrent duplicate requests.

Expected result: Provider-derived fields retain provenance and user control. Size results are tenant-scoped workflow assistance, not official SBA eligibility determinations.

Reason: External lookup and calculation results are high-overreliance features and require explicit provenance and claim limits.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-X04: Review Compliance Content Lifecycle And Development Import Boundary

Category: Compliance-content governance.

Roles: Qualified content reviewer and unauthorized tenant user.

Tenant context: Governed content scope and synthetic Tenant A; no production content changes unless separately approved.

Test data: Use a disposable synthetic clause/obligation revision with `example.invalid` source, review owner, effective date, confidence, and change reason.

Preconditions: Use an approved disposable environment. Development import must be disabled outside Development.

Page/API: Clause library, content review endpoints, and development-only import endpoint.

Steps:

1. Create or import the synthetic draft only in Development and confirm draft/rejected content is hidden from normal customer searches.
2. Add source URL, trigger/applicability logic, required action, evidence examples, risk, confidence, review owner, effective date, and last-reviewed date.
3. Move the record through review, rejection/revision, approval, publication, and retirement using authorized actions.
4. Confirm only eligible published content can be attached or generate obligations.
5. Change the source revision and verify stale approval/publication is blocked or visibly requires review.
6. Attempt review/import as an unauthorized tenant user and call the development import endpoint outside Development.

Expected result: Customer-visible compliance content is source-backed, reviewable, state-controlled, and environment-gated; draft, rejected, retired, stale, or unauthorized content cannot silently drive obligations.

Reason: Compliance content is product behavior, not static copy, and can create systemic incorrect work if governance gates are bypassed.

Security expectation: Content governance permissions and environment gates are server-authoritative. Imports must not accept client-asserted tenant scope, secrets, or prohibited content.

Evidence to capture: Content IDs/revisions, source metadata, state history, search visibility, generated-obligation eligibility, denials, and audit/trace records.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Tasks, Search, Calendar, And Notifications UAT

Current-state label: Implemented for task CRUD, signed-cursor search, calendar aggregation, renewal generation, assignment notifications, preferences, reminders, and email outbox behavior.

UAT-W01: Create, Search, Complete, Reopen, And Regenerate Tasks

Category: Compliance task management.

Roles: Compliance Manager and Contributor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tabs: Calendar and linked module views.

Steps:

1. Create Review synthetic MFA evidence with owner Devin Brooks, due T+14, and a link to the synthetic evidence item.
2. Create the T-1 overdue fixture and confirm it is visibly overdue while incomplete.
3. Search by title, owner, status, module, and date; page forward and backward with the returned cursor.
4. Modify a cursor or reuse it under Tenant B and confirm rejection without data disclosure.
5. Complete the task, reload, reopen it, and verify history and audit entries.
6. Run renewal generation twice for the same source and period.
7. Submit stale concurrent updates and invalid cross-tenant source IDs.

Expected result: Tasks remain tenant-scoped, searchable, cursor-protected, status-consistent, and idempotently generated; invalid or stale actions create no duplicate work.

Reason: Task search and recurring generation must scale without leaking tenant state or duplicating compliance work.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-W02: Verify Notification Preferences, Assignments, And Reminders

Category: Notifications and due-date reminders.

Role: Contributor receiving work; Compliance Manager assigning work.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tabs: Settings, notification bell, and Calendar.

Steps:

1. As Contributor, save the Notification preference values from Test Data and reload.
2. As Compliance Manager, assign the obligation and task to the Contributor and request assignment email.
3. Confirm one in-app notification is created and can be marked read.
4. Run due-date reminders twice for the same eligibility window.
5. Confirm reminders are deduplicated and respect preference/time-zone settings.
6. If email delivery is enabled, verify only a generic authenticated-link message is sent; otherwise mark email delivery Blocked by environment.
7. Simulate provider failure and confirm assignment persistence is not rolled back or duplicated.

Expected result: Preferences persist safely, in-app notifications and reminders are tenant-scoped and deduplicated, and external email failure is explicit without exposing record content.

Reason: Notification reliability must not create duplicate messages, cross-tenant recipients, or unsafe customer data in email.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-W03: Create And Complete A Compliance Checklist

Category: Compliance checklist templates and tenant checklists.

Roles: Compliance Manager and Auditor.

Tenant context: Active synthetic Tenant A; Tenant B identifiers are used only for negative checks.

Test data: Use a published synthetic-safe checklist template and item notes containing no customer or security-sensitive data.

Preconditions: At least one eligible checklist template is available for the tested content revision.

Tab/API: Applicable readiness panel or `/api/compliance/checklists` API.

Steps:

1. List checklist templates and inspect source/review metadata.
2. Create a tenant checklist from an eligible template.
3. Update separate items to in-progress, complete, not-applicable with rationale, and blocked with owner/follow-up.
4. Reload and compare progress calculations with item states.
5. Attempt a stale update, unsupported status, missing rationale, draft template, and Tenant B checklist/item ID.
6. Repeat item mutations as Auditor.

Expected result: Checklist instances and progress remain tenant-scoped, source-linked, role-controlled, version-safe, and auditable; completion never becomes a certification or official compliance claim.

Reason: Checklist percentages can be materially misleading if item state, rationale, source version, and authorization are not enforced.

Security expectation: Server-side permission and tenant ownership govern every template use and item mutation; invalid actions leave checklist progress and audit history unchanged.

Evidence to capture: Template/source metadata, checklist and item states, progress before/after, denials, stale-version response, and audit events.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Extended Evidence And Classification UAT

Current-state label: Implemented for metadata, private file lifecycle, classification, review, evidence requests, classified notes, classification review queue, and support escalation. Scanner/storage availability is environment-dependent.

UAT-E01: Upload, Download, Replace, And Delete An Allowed Evidence File

Category: Evidence file lifecycle and malware scanning.

Role: Contributor; Auditor for read-only checks.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: Evidence.

Prerequisite: UAT-10 and UAT-11 are complete.

Steps:

1. Request an upload intent for `uat-mfa-summary.txt` with FCI classification and the required acknowledgement.
2. Upload the harmless UTF-8 bytes and confirm validation and malware scanning complete before the file becomes downloadable.
3. Download as an authorized user and compare the SHA-256 hash with the source fixture.
4. Replace it with `uat-mfa-summary-v2.txt`; verify the new version and history without losing the prior metadata trail.
5. Attempt unsupported type, oversized payload, extension/content mismatch, unsafe filename, scanner unavailable, and Tenant B evidence ID.
6. Delete the active file with a reason and confirm later download fails safely while audit/history remains.
7. As Auditor, attempt upload, replacement, and deletion directly.

Expected result: Only validated, allowed, scanned bytes enter private storage. File lifecycle, versions, hashes, authorization, and audit remain consistent; rejected files leave no durable object or success event.

Reason: File handling is the highest-risk No-CUI boundary and must be verified beyond metadata creation.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-E02: Review Evidence And Enforce Separation Of Responsibility

Category: Evidence review and approval.

Roles: Contributor submitter, authorized reviewer, Auditor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it Passed.

Tab: Evidence.

Steps:

1. Create a new synthetic evidence item as Contributor and submit it for review.
2. Attempt to approve it as the submitter where separation of responsibility is configured.
3. Approve it as the authorized reviewer with review date and notes.
4. Edit the evidence metadata and confirm prior approval resets when the implemented contract requires re-review.
5. Return it for correction, resubmit, and approve again.
6. Attempt review against a Tenant B item and repeat a stale decision.

Expected result: Review state, reviewer identity, review date, notes, reset behavior, and history are visible and tenant-scoped. Unauthorized, self-review-restricted, stale, and cross-tenant decisions fail without false approval.

Reason: Evidence status is relied on by reports and readiness calculations and cannot be a cosmetic field.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-E03: Complete An Evidence Request Workflow

Category: Evidence requests and collaboration.

Roles: Compliance Manager requester, Contributor respondent, reviewer.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it Passed.

Tabs: Evidence, Calendar, and notification bell.

Steps:

1. Create `Synthetic annual access-review evidence request` for the Contributor with due T+14 and the synthetic control/obligation scope.
2. Confirm it appears in the request list, calendar, and recipient notification.
3. Send reminders twice and verify deduplication.

4. As Contributor, link the allowed evidence item and submit the request.
5. As reviewer, return it with a correction note, then accept the corrected resubmission.
6. Attempt submission with prohibited, expired, missing, or Tenant B evidence.

Expected result: Request ownership, due date, evidence link, submission/review history, reminders, calendar, and audit remain consistent; ineligible evidence cannot satisfy the request.

Reason: Evidence collaboration must preserve the same eligibility and tenant rules as direct report generation.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-E04: Verify Classification Review, Classified Notes, And Escalation

Category: No-CUI classification governance.

Roles: Contributor, classification reviewer, and support administrator.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tabs: `Evidence` and `Settings`.

Steps:

1. Create the allowed synthetic classified note with FCI and a reason.
2. Edit it and confirm revision history records classification metadata without storing note text in audit metadata.
3. Attempt `Unknown`, `Prohibited`, `Cui`, `classified`, `export-controlled`, and `contradictory` classifications using only harmless synthetic bytes/text.
4. Open the classification review queue and record a review decision on an eligible item.
5. Create a CUI support escalation using the synthetic summary, change owner/severity/status, and resolve it with closure notes.
6. Search escalation history and audit; attempt Tenant B IDs and unauthorized updates.

Expected result: Allowed notes retain versioned classification; prohibited/uncertain cases are blocked or routed to review/escalation before downstream use. Audit omits raw content, and tenant/role boundaries hold.

Reason: Free-text fields, filenames, and metadata can carry sensitive content even when file bytes are blocked.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Extended CMMC, SPRS, And SSP UAT

Current-state label: Implemented as internal readiness workflow. It does not certify CMMC compliance, create an assessor determination, authorize CUI, or submit to SPRS.

UAT-C05: Create Gaps And Close A POA&M Item

Category: CMMC gaps, remediation, accepted risk, and closure.

Role: Compliance Manager; reviewer for closure.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: CMMC.

Steps:

1. Open the Level 1 assessment and inspect the gap projection for AC.L1-3.1.1.
2. Create or update a POA&M with High severity, Security owner, T+14, remediation plan, and synthetic compensating-control text.
3. Attempt closure without required notes/evidence and confirm validation.
4. Link the reviewed evidence item, enter closure notes, and close the item.
5. Reopen or update only where the lifecycle permits and confirm task/calendar synchronization.
6. Submit an expired/prohibited/Tenant B evidence link and a stale concurrent closure.

Expected result: Gap, POA&M, task, evidence, owner, due date, compensating control or accepted-risk metadata, closure, and history remain traceable and conflict-safe.

Reason: Remediation closure must be evidence-based and auditable rather than a misleading status toggle.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-C06: Record Affirmation And Responsibility Assignments

Category: CMMC affirmation preparation and responsibility matrix.

Role: Compliance Manager; Owner where required.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: CMMC.

Steps:

1. Create a synthetic affirmation-preparation record with owner and due date.
2. Update its status and review metadata without entering an external SPRS submission claim.
3. Assign representative controls as Customer, Provider, and Shared responsibility.
4. Export the responsibility matrix and compare every assignment with the UI/API.
5. Attempt unauthorized, stale, and Tenant B updates and exports.
6. Confirm assignment does not grant record access or change the tenant data-handling mode.

Expected result: Affirmation preparation and responsibility assignments are tenant-scoped, versioned, auditable, and accurately exported without representing external affirmation or authorization.

Reason: Responsibility metadata guides work allocation but is not an access-control or certification decision.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-C07: Calculate And Review A Draft SPRS Score

Category: SPRS readiness calculation.

Role: Compliance Manager; Auditor for read-only review.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: CMMC.

Steps:

1. List available scoring rule sets and identify the published, effective source revision.
2. Attempt a calculation with a draft, retired, or ineffective rule set.
3. Calculate the draft score for the synthetic assessment using an idempotency key.
4. Repeat the same request and confirm one immutable calculation result.
5. Compare maximum score, deductions, unresolved gaps, control status, and rationale with assessment data.
6. Add a review note without mutating the recorded calculation inputs.
7. Attempt Tenant B assessment and unauthorized calculation access.

Expected result: Only a published effective ruleset creates an immutable, source-hashed draft calculation. The result is labeled readiness information and is not automatically submitted to SPRS.

Reason: A score can materially mislead users unless its rules, inputs, limitations, and immutability are explicit.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-C08: Build, Approve, Package, And Share An SSP Narrative

Category: SSP internal-review workflow.

Roles: Compliance Manager, reviewer, Owner for external-share approval, Auditor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: CMMC; SSP panels.

Steps:

1. Create SSP section 3.1 `Access Control` and link governed synthetic sources.
2. Generate a deterministic source-backed narrative; request external AI assistance and confirm it fails closed unless a separately approved provider exists.
3. Edit the draft with explicit FCI classification and submit it for review.
4. Change or expire a source and attempt approval; confirm approval is blocked.
5. Restore an eligible source state, approve a replacement narrative, and verify the prior approved narrative is superseded rather than overwritten.
6. Generate Internal SSP review package - synthetic with eligible evidence and POA&M records.
7. Attempt packaging with expired, prohibited, unknown, CUI, missing, or Tenant B input.
8. Attempt sharing before and after package-specific Owner approval.

Expected result: SSP sections, narratives, sources, reviewer metadata, immutable packages, lifecycle history, and share approval are tenant-scoped and auditable; any ineligible selection rejects the entire package.

Reason: SSP output aggregates many records and requires stronger immutability and eligibility checks than a simple text export.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Subcontractor And Partner Collaboration UAT

Current-state label: Implemented for subcontractor profiles, flow-downs, supplier obligations, evidence requests, reports, and shared-package lifecycle. External portal identity and approved-package catalogs remain partially implemented.

UAT-S01: Create And Review A Subcontractor Profile

Category: Subcontractor profile and risk status.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tab: Subcontractors.

Steps:

1. Create Potomac Synthetic Services LLC with the synthetic UEI/CAGE/contact values.
2. Add role, workshare, small-business metadata, insurance/NDA dates, CMMC readiness metadata, and an FCI only data-access flag.
3. Save and confirm risk/expiry indicators are derived from the current values.
4. Use optional SAM lookup and apply only intended synthetic fields when the provider is configured.
5. Update one value and verify provenance and audit.
6. Attempt invalid date/value combinations and Tenant B identifiers.

Expected result: The tenant-scoped profile, source/provenance, risk indicators, expiration data, and history persist without making an eligibility or compliance determination.

Reason: Subcontractor metadata can influence prime decisions and must remain factual, sourced, and clearly non-determinative.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-S02: Track Flow-Downs And Supplier Obligations

Category: Flow-down clause and supplier work tracking.

Role: Compliance Manager; Advisor when explicitly assigned.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tab: Subcontractors.

Steps:

1. Attach the reviewed FAR 52.204-21 flow-down to the synthetic subcontractor and contract.
2. Move status from Required to Sent to Acknowledged using reasons and dates.
3. Bulk-generate supplier obligations twice from eligible flow-downs.
4. Confirm one obligation exists with owner, due date, status, source clause, contract, and subcontractor links.
5. Update owner/status and link eligible evidence.
6. Attempt a draft clause, Tenant B contract, unrelated evidence, unauthorized role, and stale transition.

Expected result: Flow-down and supplier-obligation lifecycle is source-linked, idempotent, tenant-scoped, and audited; invalid references create no partial obligations.

Reason: Prime-contractor flow-down work must retain its legal-source context without becoming unsupported legal advice.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-S03: Request Evidence And Generate A Subcontractor Report

Category: Subcontractor collaboration and reporting.

Roles: Compliance Manager, external contact without membership, Auditor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tabs: Subcontractors and Reports.

Steps:

1. Create an evidence request for the synthetic subcontractor with due date T+14 and allowed scope.
2. Record submission/review actions through the implemented tenant workflow.
3. Verify an external contact field alone does not grant tenant access or mutation permission.
4. Generate a subcontractor compliance report and compare profile, flow-down, obligation, evidence-request, risk, and expiry values with the UI/API.
5. Attempt unauthorized generation/export and Tenant B scope.
6. Confirm report wording avoids eligibility, certification, or compliance determinations.

Expected result: Collaboration state remains tenant-controlled and the report accurately snapshots authorized metadata without granting access from contact affiliation.

Reason: Partner records and reports combine identity, contract, and evidence data and are high-value cross-tenant targets.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-S04: Exercise Shared Portal Package Lifecycle

Category: External package access and lifecycle.

Roles: Tenant Admin, explicitly invited external reviewer, unrelated external user.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Settings`; portal API/test route where no production external UI exists.

Steps:

1. Share the approved synthetic package with an eligible invitation and future expiry.
2. As the invited reviewer, view, download, and comment; confirm the source package is unchanged.
3. Attempt access with an unrelated, expired, revoked, or Tenant B invitation and with a draft/prohibited package.
4. Revoke the share and retry a previously opened link.
5. Reissue it and confirm the new active share supersedes only eligible predecessors.
6. Run expiration/reminder maintenance twice and inspect activity history.
7. Archive the terminal lifecycle record and attempt concurrent stale actions.

Expected result: Request-time checks enforce package, invitation, tenant, status, and expiry on every access. Lifecycle and activity are append-only; cached links fail after revocation or expiration.

Reason: External sharing must not depend on UI state or background workers for security enforcement.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

SAM.gov SPR Preparation UAT

Current-state label: Implemented for internal preparation, governed schema, immutable packages, export, and user-entered receipt history. Automated SAM.gov submission is disabled and outside scope.

UAT-SPR01: Configure SPR Applicability And Schedule

Category: Subcontracting Plan Reporting applicability preparation.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Contracts`; SPR applicability panel.

Steps:

1. Create an applicability record for `DEMO-NC-26-0007` using the synthetic reporting role and period.
2. Add source, rationale, effective dates, and reporting schedule.
3. Attempt activation without required source/rationale and confirm rejection.
4. Activate the complete record and verify calendar/task generation.
5. Update or deactivate it and confirm schedule synchronization.
6. Retry activation and submit a Tenant B contract or stale version.

Expected result: Applicability remains an explicit source-backed user workflow; activation is validated, tenant-scoped, auditable, and does not represent a legal reporting determination.

Reason: Report preparation must not begin from an unreviewed or cross-tenant applicability assumption.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-SPR02: Enter, Import, Review, And Remediate SPR Data

Category: SPR report data and governed schema.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Contracts`; SPR report-data panel.

Steps:

1. Inspect the current governed schema profile and download the exact import template.
2. Create a manual row using synthetic UEI, PIID, category, period, and 125000 whole dollars.
3. Review and accept the row, then edit it and confirm review resets.
4. Import `spr-uat-valid.csv`, then a mixed invalid/duplicate/oversized file.
5. Confirm row-level errors identify safe fields without executing spreadsheet formulas.
6. Open remediation for an intentionally incomplete legacy row and apply suggested UEI/PIID values only after review.
7. Attempt Tenant B subcontractor/evidence references and concurrent duplicate rows.

Expected result: Manual and imported rows follow the same schema, identity, duplicate, evidence, review, and tenant rules. Legacy rows are not silently promoted to package eligibility.

Reason: Bulk imports and migrated rows are common bypass paths around normal validation and review.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-SPR03: Generate, Export, And Record A Manual SPR Receipt

Category: SPR package lifecycle and submission metadata.

Role: Compliance Manager with export permission; Auditor for read-only review.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Contracts`; SPR packages panel.

Steps:

1. Inspect package eligibility and correct all blocking fields using synthetic data.
2. Generate a package, begin review, approve it, and export HTML/JSON.
3. Compare schema profile, row counts, totals, evidence references, exceptions, version, and hash with the source rows.
4. Record manual receipt `SAM-SPR-UAT-RECEIPT-001` and then create a correction that supersedes it.
5. Supersede and archive the package using reasons; retry stale lifecycle actions.
6. Check submission capability and call the submit route in the approved test environment.
7. Confirm the capability is unavailable, the call fails closed, and no outbound SAM.gov interaction or acceptance claim occurs.

Expected result: The package is an immutable tenant-scoped preparation snapshot; receipt history is user-asserted and append-only; automated submission remains unavailable.

Reason: FeDril must distinguish internal preparation and user-entered metadata from official external submission or verification.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Labor Readiness UAT

Current-state label: Implemented. Labor applicability, classification, dashboard/report persistence, sensitive-field redaction, authorization, and migration checks pass focused local verification. PostgreSQL-backed execution remains an environment-dependent UAT gate.

UAT-L01: Record Labor Applicability And Wage Evidence

Category: Labor applicability readiness.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Contracts`; Labor applicability panel.

Steps:

1. Create SCA applicability using the synthetic source, rationale, dates, and responsible owner.
2. Attempt activation with missing source/rationale and confirm rejection.
3. Activate the complete record and confirm one linked compliance task.
4. Upload synthetic wage-determination evidence through the standard acknowledgement, classification, scanning, and private-storage path.
5. Update status and verify task, evidence, history, and audit linkage.
6. Attempt a Tenant B clause/evidence reference and prohibited classification.

Expected result: Labor applicability is explicit, source-backed, tenant-scoped, and linked to the shared evidence/task workflow without accepting payroll or real employee data.

Reason: The module organizes readiness work and must not be presented as a legal wage determination.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-L02: Classify And Reclassify A Synthetic Employee Assignment

Category: Labor categories and employee classifications.

Roles: Compliance Manager with sensitive-data permission; ordinary Viewer without it.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Contracts`; Labor classification panel.

Steps:

1. Create `Help Desk Specialist - Synthetic` with the test wage/fringe values, source, and effective period.
2. Assign `UAT-EMP-0001` to that category and contract for T through T+180.

3. Attempt an overlapping active assignment and confirm rejection.
4. View the assignment as an authorized sensitive-data role, then as an ordinary permitted viewer.
5. Confirm employee name/email are redacted for the ordinary viewer.
6. Reclassify with a reason, verify prior/new category history, and confirm review resets to pending.
7. Approve/return the review, edit, deactivate, and attempt stale or Tenant B actions.

Expected result: Category and assignment rules, overlap prevention, sensitive-field redaction, reclassification history, reviewer meta-data, and audit are enforced and tenant-qualified.

Reason: Employee-linked records require a separate sensitive-field permission and append-only classification history.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-L03: Verify Labor Dashboard And Immutable Report

Category: Labor dashboard and report.

Role: Compliance Manager and Auditor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tabs: `Contracts` and `Reports`.

Environment dependency: Execute against the exact release commit only after CI applies its migrations and the focused plus PostgreSQL real-stack tests pass.

Steps:

1. Attach the successful build, migration, and labor test evidence; otherwise mark this case `Blocked by environment`.
2. Filter `/labor/dashboard` by contract, applicability, category, review status, and effective date.
3. Compare counts and redacted/sensitive views with source records.
4. Generate the labor compliance report and verify immutable scope, source references, review state, classifications, and disclaimer language.
5. Attempt unauthorized export, Tenant B scope, prohibited evidence, duplicate generation, and stale data.

Expected result: After the prerequisite is satisfied, dashboard and report data reconcile and remain tenant-scoped, permission-aware, immutable, and explicitly non-determinative. Until then, this case cannot pass.

Reason: Route presence alone is insufficient; retain focused test, migration, and deployed-environment evidence with the UAT record.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Policies, Suggested Guidance, And Guarded Assistance UAT

Current-state label: Implemented for templates, generated-policy lifecycle, suggested obligations, citations, logging, and human review. External AI providers are not enabled by default.

UAT-AI01: Generate And Review A Policy From A Template

Category: Policy templates and generated policy lifecycle.

Roles: Compliance Manager and authorized reviewer.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: API-backed workflow or exposed policy panel.

Steps:

1. List published templates and select `Access Control Policy - Synthetic` with source/review metadata.
2. Generate a draft using only synthetic facts and explicit FCI classification.
3. Confirm placeholders, source references, draft label, creator, and revision are visible.
4. Edit the draft and submit for review.
5. Attempt approval with unresolved placeholders, changed/expired source, unknown/prohibited classification, and self-review where separation is required.
6. Correct the draft, approve it with an authorized reviewer, and inspect revisions and derived evidence links.
7. Attempt Tenant B read/update and stale review actions.

Expected result: Generated policy content remains draft until an eligible human approval; sources, classification, revisions, review metadata, and audit remain tenant-scoped.

Reason: Generated compliance text must never become authoritative solely because a generation request succeeded.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-AI02: Verify Citation, Logging, And Fail-Closed Assistant Behavior

Category: Guarded suggestions and AI-assistance boundary.

Role: Compliance Manager.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: API-backed suggestion or assistance workflow.

Steps:

1. Request a suggestion using governed synthetic source identifiers.
2. Confirm the draft output contains human-readable citations and provenance.
3. Review, revise, approve, reject, and escalate separate synthetic outputs.
4. Request an unsupported answer, a source-free answer, and a prompt containing a synthetic sensitive-data marker.
5. Inspect logs/audit and confirm they contain identifiers and safe metadata, not full prompts, generated sensitive text, tokens, or secrets.
6. Request external AI processing when the provider is disabled.
7. Attempt Tenant B sources and reviewer actions.

Expected result: The workflow is source-bound, draft-only, review-gated, safely logged, tenant-scoped, and fails closed when sources or approved providers are unavailable.

Reason: AI-like assistance increases overreliance and data-leak risk unless provenance, provider posture, and human review remain explicit.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Extended Reporting And Audit UAT

Current-state label: Implemented for current reports, report detail, archive/restore, authorized exports, PDF jobs, and audit-log/CUI-dimension export. Renderer/storage and database transaction checks require the configured environment.

UAT-R01: Verify Report, Export, Archive, Restore, And PDF Lifecycle

Category: Reporting and immutable artifacts.

Roles: Compliance Manager, user with `ExportReports`, Auditor, Contributor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it Passed.

Tab: `Reports`.

Steps:

1. Generate compliance status, evidence package, CMMC readiness, SPRS readiness, subcontractor, and eligible labor reports.
2. Open each report, reload, and compare scope, counts, owners, dates, sources, classifications, review state, and limitations with UI/API records.
3. Export each supported report and verify hash, filename safety, formula neutralization, tenant metadata, and row counts.
4. Archive and restore one report with explicit reasons; confirm content/hash does not change and history appends.
5. Request a PDF, poll the job, download it, and visually inspect title, scope, pagination, readable text, and disclaimers.
6. Retry generation/export/PDF requests and simulate renderer/storage failure.
7. Attempt every generation, archive, restore, export, and download as Auditor/Contributor and with Tenant B IDs.

Expected result: Reports are immutable, tenant-scoped snapshots; lifecycle changes preserve content; exports/PDFs reconcile and are independently authorized; failures create no unauthorized or partial artifact.

Reason: Reports and exports are a concentrated disclosure and overclaiming boundary.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-R02: Verify Audit Search, Export, Append-Only Behavior, And Rollback

Category: Audit logging and transaction integrity.

Role: Owner/Advisor with `ViewAuditLog`; unauthorized Contributor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it Passed.

Tab: `Settings`; API harness for failure injection.

Steps:

1. Filter audit history by actor, action, entity, date, classification, and trace ID; page forward/backward.
2. Compare representative security-sensitive mutations with their audit events and request trace IDs.
3. Export the authorized CUI/classification audit projection using synthetic metadata only.
4. Attempt direct update/delete of audit records and unauthorized/cross-tenant reads or exports.
5. Inject an audit-writer failure for a compliance-relevant relational mutation in an approved disposable environment.
6. Confirm the business mutation rolls back with no success audit, job, notification, file, or external side effect.
7. Where an external provider uses an outbox, verify the documented atomic boundary and retry behavior separately.

Expected result: Audit history is append-only, tenant-scoped, searchable, authorized, safely exportable, and transactionally consistent with protected relational mutations.

Reason: An audit trail that can be altered, omitted, or separated from the business write cannot support accountability.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

No-CUI Governance And Operational Readiness UAT

Current-state label: Implemented as notices, acknowledgements, readiness evidence, approval checklists, shared-responsibility acknowledgements, support escalations, and security/technical/incident-readiness records. None authorizes production CUI by itself.

UAT-N01: Verify Contextual Data-Handling Notices And Gating

Category: No-CUI notice and acknowledgement enforcement.

Roles: Contributor and Owner.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in Test Data and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case Blocked by environment; do not mark it Passed.

Tabs: Settings, Contracts, Evidence, and Reports.

Steps:

1. Retrieve published notices for Onboarding, Contract upload, Evidence upload, Extraction job, Classified note, Report generation, and Support contexts.
2. Attempt each gated action before acknowledgement.
3. Acknowledge one context and confirm unrelated contexts remain independently gated where required.
4. Change the published notice revision or tenant mode in an approved fixture and retry the prior acknowledgement.
5. Attempt forged user/tenant IDs and Tenant B acknowledgement IDs.
6. Confirm notice and acknowledgement history identifies version, context, actor, and timestamp without storing sensitive text.

Expected result: Current contextual acknowledgements are server-validated before protected actions; stale, missing, forged, and cross-tenant acknowledgements do not authorize processing.

Reason: A single generic checkbox cannot safely govern distinct upload, extraction, report, and support contexts over time.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-N02: Exercise CUI-Readiness Checklist And Evidence Without Authorizing CUI

Category: Future CUI-readiness governance records.

Roles: Owner, authorized independent approver, ordinary Tenant Admin.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Settings`.

Steps:

1. Create a synthetic readiness checklist and update items with owners, dates, sources, and evidence references.
2. Record `Synthetic restore rehearsal record` as readiness evidence after acknowledging the current notice.
3. Attempt submission with incomplete items, expired/unknown/prohibited evidence, open critical gaps, or stale versions.
4. Submit a complete synthetic checklist and attempt approval as the same actor or an unauthorized Tenant Admin.
5. Verify any positive transition remains readiness metadata and does not enable real upload, claim FedRAMP/CMMC authorization, or silently change `NoCui`.
6. Reject/supersede the fixture with reasons and inspect history/audit.

Expected result: Checklist/evidence lifecycle is tenant-scoped, source-backed, reviewable, and fail-closed. No checklist alone authorizes real CUI or creates a certification/government claim.

Reason: Readiness workflow records must remain separate from actual deployment authorization and operating evidence.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-N03: Verify Security, Technical, And Incident Readiness Records

Category: Operational readiness evidence.

Roles: Authorized owner/reviewer and read-only Auditor.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Settings`; Security incident readiness panel.

Steps:

1. Create or update synthetic security-review, technical-readiness, and incident-readiness records using no production security detail.
2. Link only eligible same-tenant readiness evidence.
3. Attempt approval with missing evidence, unresolved critical findings, expired sources, or stale version.
4. Approve complete synthetic records using the authorized reviewer.
5. Compare summary and history projections with individual records and audit events.
6. Attempt unauthorized and Tenant B reads, writes, and approvals.

Expected result: Readiness records and approvals are evidence-linked, role-separated where configured, version-safe, tenant-scoped, and auditable without exposing sensitive incident data.

Reason: Operational-readiness screens must not become a repository for secrets or a self-attested production-security claim.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-N04: Acknowledge Shared Responsibility And Resolve A Support Escalation

Category: Shared responsibility and support escalation.

Roles: Owner, Contributor, support administrator.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Tab: `Settings`.

Steps:

1. Open the published shared-responsibility matrix and record its version/source.
2. As Owner, acknowledge the current matrix and confirm history.
3. Change the published revision in an approved fixture and verify the prior acknowledgement is visibly stale.
4. Create the synthetic support escalation, assign owner/severity/due date, change status, and resolve it with closure notes.
5. Generate or view the escalation report and compare counts/history.
6. Attempt Contributor administration, Tenant B IDs, duplicate resolution, and stale updates.

Expected result: Matrix acknowledgements and escalation lifecycle are versioned, tenant-scoped, role-controlled, and audited; neither represents legal acceptance of CUI processing.

Reason: Customer responsibility and suspected-CUI response require explicit ownership and history without weakening the No-CUI boundary.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

UAT-N05: Seed, Verify, And Remove A Synthetic Demo Dataset

Category: Synthetic DemoSandbox lifecycle.

Roles: Authorized Owner/Admin in an approved disposable DemoSandbox; unauthorized user.

Tenant context: Dedicated synthetic DemoSandbox only. Never run seed/delete against a customer or production tenant.

Test data: Use the server-provided synthetic dataset manifest and expected safe-content approvals.

Preconditions: Confirm the exact tenant ID, DemoSandbox posture, environment permission, backup/cleanup plan, and absence of customer records.

Tab/API: Demo dataset precheck, seed, inspect, and delete endpoints; affected workspace modules.

Steps:

1. Retrieve the synthetic dataset manifest and run the precheck.
2. Confirm every included record is synthetic, classified for the demo posture, and approved for the configured dataset revision.
3. Seed once and reconcile created profile, contracts, obligations, evidence metadata, tasks, assessments, reports, and audit events.
4. Seed again and confirm idempotent or explicitly conflict-safe behavior without duplicates.
5. Attempt seed/delete as an unauthorized role, in Tenant A NoCui, and with a Tenant B ID.
6. Delete only the recorded seeded dataset from the disposable DemoSandbox and confirm unrelated records/audit history are preserved.

Expected result: Synthetic demo lifecycle is explicitly gated, tenant-scoped, revision-controlled, repeat-safe, and removable without broad deletion or any inference that DemoSandbox permits real CUI.

Reason: Seed utilities can bypass normal workflows or destroy data if tenant, environment, classification, and ownership checks are weak.

Security expectation: The server must reject customer/production/unauthorized/cross-tenant use and must never log or generate real sensitive content, credentials, or secrets.

Evidence to capture: Tenant/mode precheck, manifest/revision, created-record counts and IDs, repeat result, denials, scoped cleanup proof, and audit history.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Enterprise And Regulated-Deployment Boundary UAT

Current-state label: Planned or partially implemented scaffolding. Not applicable to No-CUI MVP approval. Existing SAML, SSO, SCIM, government-cloud, FedRAMP, enclave, customer-managed-key, and regulated-provisioning APIs/tests do not prove an activated provider, assessed boundary, certification, authorization, or real-CUI capability.

UAT-O01: Confirm Future Enterprise Features Do Not Alter MVP Claims Or Access

Category: Future-scope fail-closed posture.

Roles: Normal Tenant Admin and authorized engineering test operator in a disposable synthetic environment.

Tenant context: Use the tenant or platform/public context named in this case. When no tenant is named, use active synthetic Tenant A; use only recorded Tenant B identifiers for negative isolation steps.

Test data: Use the exact synthetic fixtures in `Test Data` and the unique run ID. Never substitute customer or sensitive data.

Preconditions: Complete referenced prerequisite cases and verify required environment/provider dependencies. If a dependency is unavailable, mark only the affected step or case `Blocked by environment`; do not mark it `Passed`.

Page/API: Enterprise endpoints have no general MVP acceptance UI unless explicitly enabled for a separately governed release.

Steps:

1. Confirm normal No-CUI users are not presented with claims that SAML/SCIM, government cloud, FedRAMP authorization, CUI enclave, or customer-managed keys are active.
2. Confirm a normal Tenant Admin cannot create or approve regulated environments, FedRAMP packages, enclave access, support access, emergency access, or key-policy activation.
3. Verify configured provider-dependent operations fail closed when required infrastructure or approval evidence is absent.
4. Confirm any synthetic readiness package says internal readiness only and never reports FeDril as FedRAMP-authorized, CMMC-certified, government-approved, or approved for real CUI.
5. Do not execute destructive provisioning, real identity-provider changes, real keys, or real CUI operations as part of MVP UAT.
6. Record this case `Not applicable` for behavior beyond the fail-closed/claim check unless a separate regulated-release UAT is approved.

Expected result: Future scaffolding remains inaccessible or safely limited in the No-CUI MVP and cannot create positive authorization wording or real-CUI processing capability.

Reason: Code presence is not operational authorization, and future endpoints must not silently expand the accepted product boundary.

Security expectation: Server-side authentication, active tenant membership, tenant isolation, and the action-specific permission remain authoritative. Rejected or cross-tenant actions must not expose protected metadata or create an unauthorized business record, success audit, file, job, notification, token, or external effect.

Evidence to capture: Role and tenant context, relevant UI state, sanitized request and response with HTTP status and trace ID, record identifiers, before-and-after state, audit event where required, and no-side-effect proof for negative steps.

Execution record: Status: ☐ Pass ☐ Fail ☐ Blocked by environment ☐ Not applicable ☐ Not run | Defect: _____ | Tester: _____ | Date/time: _____ | Evidence location: _____ | Notes: _____

Acceptance Exit Criteria

The UAT passes only if all of these conditions are true:

1. Only an authorized Platform Operator can open and submit `Tenant onboarding`; a normal tenant Owner receives an authorization denial.
2. Pilot and Paid onboarding both begin as `PendingOwnerAcceptance`, `PendingActivation`, and `NoCui`, with the correct mode-specific fields.

3. The exact invited Owner identity activates the Pilot as `Trialing` and the Paid tenant as `Active`. If invitation delivery is unavailable, record `UAT-T03` and the tenant-onboarding category as `Blocked by environment, not Passed`.
4. Identical onboarding retries are idempotent, conflicting or duplicate requests are rejected, and pending cancellation preserves an archived, audited record.
5. Tenant mode is `NoCui`.
6. The synthetic company profile can be saved as a draft, rejects premature completion, and reaches `Complete` and 100% only after required fields are present.
7. Contributor, Auditor, and Advisor can view but cannot modify the company profile, and cross-tenant profile data is not disclosed.
8. Contract `DEMO-NC-26-0007` exists and displays the expected metadata.
9. The synthetic deliverables persist, appear on the calendar, display overdue state correctly, and retain authorized status updates.
10. Contributor and Auditor cannot mutate deliverables, while Advisor behavior matches the current `ManageContracts` permission, and cross-tenant deliverable data is not disclosed.
11. Published clauses are found and attached with source references.
12. At least one obligation is generated or visible from the attached clause mapping.
13. Obligation status and owner assignment can be updated by an authorized role.
14. Allowed synthetic FCI evidence metadata can be created.
15. CUI classification or upload is blocked in No-CUI mode.
16. A CMMC readiness assessment is created and shows owner, dates, progress, and control-readiness information.
17. CMMC control readiness preserves source metadata and traceability warnings.
18. A CMMC POA&M item is created and appears as CMMC remediation work on the calendar.
19. A current report, evidence package, and CMMC readiness report artifact are generated.
20. Report language avoids certification, legal, compliance, government-approval, and audit-readiness overclaims.
21. Audit history shows the tested onboarding, profile, contract, deliverable, clause, evidence, CMMC, and report events.
22. Unauthorized roles cannot mutate restricted onboarding or tenant workflow records.
23. Public demo input is safely validated, platform follow-up remains separate from tenant privileges, and platform customer aggregation is accessible only with explicit platform permission.
24. Invitations, membership removal, tenant switching, logout/session expiry, and subscription transitions preserve authorization and isolation.
25. Dashboard values reconcile with module/API data and never display stale Tenant A data after switching to Tenant B.
26. Extraction, applicability, suggested obligations, size assistance, provider results, and compliance-content lifecycle preserve source, publication, environment, and human-review boundaries.
27. Tasks, checklists, cursors, recurring generation, notifications, and reminders are tenant-scoped, source-aware, and duplicate-safe.
28. Allowed evidence files are scanned and privately stored; prohibited, invalid, or cross-tenant files leave no durable object.
29. Evidence review, requests, classified notes, classification review, and support escalation preserve role, tenant, and audit boundaries.
30. CMMC gaps, POA&M closure, affirmation preparation, responsibility assignments, SPRS calculations, and SSP packages remain source-backed readiness records without certification or external-submission claims.
31. Subcontractor profiles, flow-downs, supplier obligations, evidence requests, reports, and portal shares prevent unauthorized partner access.
32. SPR data follows the governed schema and immutable package lifecycle; the submit capability remains unavailable and user-entered receipts are not represented as verification.
33. Labor applicability and classification pass source, tenant, overlap, redaction, review, and audit checks; `UAT-L03` remains `Blocked` until the current labor report implementation compiles and is verified.
34. Policies and assistance outputs remain draft, cited, classified, logged safely, and human-reviewed; unavailable external AI fails closed.
35. Reports, exports, archive/restore, PDFs, and audit exports reconcile with source records and enforce independent authorization.
36. No-CUI notices, readiness evidence, checklists, shared responsibility, incident-readiness, and synthetic demo lifecycle do not authorize real CUI, affect customer tenants, or create certification/government claims.
37. No P0 or P1 defects remain open. Any P2 exception has an owner, mitigation, expiry, and approvals from Product and Security.
38. Every applicable test has observable evidence, and every `Blocked` or `Not applicable` result has a documented reason and approver.

Mandatory Negative And Security Tests

Run these tests even when the positive workflow passes:

1. Repeat every tenant-scoped read, detail, mutation, search, page, report, export, download, background job, and linked-record action with a Tenant B identifier while authenticated to Tenant A.
2. Repeat every protected mutation using Auditor, Contributor, deactivated-member, unaccepted-invitee, and unauthenticated/expired-session contexts as applicable.
3. Attempt direct API calls when the UI hides or disables controls. UI state is not authorization evidence.
4. Submit missing, malformed, oversized, duplicate, stale, replayed, conflicting, and unsupported values. Verify no partial business state, success audit, job, notification, token, file, or external call is created.
5. Use only harmless synthetic content to test `Cui`, `Unknown`, `Prohibited`, classified, export-controlled, ITAR, unsafe filename, extension mismatch, formula injection, markup injection, and scanner-unavailable cases.

6. Inspect UI, API responses, logs, audit metadata, exported files, PDFs, notifications, and provider payloads for tenant names, raw content, stack traces, credentials, secrets, tokens, cookies, connection strings, signed URLs, or internal implementation detail.
7. Exercise stale-version and concurrent actions for approval, closure, archive/restore, reissue, package lifecycle, assignment, classification, and subscription transitions.

Any cross-tenant access or inference, authentication bypass, unauthorized role action, No-CUI bypass, required security-audit omission, incorrect report/export authorization, data loss/corruption, secret exposure, or materially misleading readiness result blocks UAT approval and pilot onboarding.

Smoke Test Sequence

Execute after every deployment in this order:

1. GET /health returns 200, the service identifies the No-CUI posture, and dependency status is explicit.
2. Public landing page loads and displays FeDril branding and No-CUI claim limits.
3. Valid identity reaches the workspace; invalid or expired identity cannot.
4. Tenant A selection loads Dashboard and all ten navigation routes without Tenant B data.
5. Auditor can read one authorized report and receives 403 on a direct report-generation request.
6. Tenant A receives tenant-safe 404 or the established 403 for one Tenant B contract, evidence, report, and export request.
7. Allowed synthetic evidence metadata/file path succeeds; Cui classification using harmless bytes is blocked before storage.
8. Minimal contract -> clause -> obligation -> task/evidence -> report -> audit path succeeds.
9. Report export/download is private and authorized; audit history contains the successful security-sensitive actions.

Stop the smoke run on authentication, tenant, RBAC, No-CUI, file-storage, report/export, audit, data-integrity, or secret-leak failure.

Regression Test Execution

Regression ID	Command or activity	Expected result	Result / Evidence
REG-01	dotnet test tests/ Gccs.Api.Tests/Gccs. Api.Tests.csproj	Zero failures; record counts, duration, skipped tests, database provider, and environment differences	
REG-02	Repeat PostgreSQL isolation, concurrency, audit rollback, report, SPR, SSP, portal, notification, and labor tests with GCCS_TEST_POSTGRES_CONNECTION	Real relational constraints and transaction behavior pass	
REG-03	npm run lint:web	Zero lint errors	
REG-04	npm run test:web	Zero failures; record count and duration	
REG-05	npm run build:web	Production frontend build succeeds	
REG-06	Validate migrations against clean and upgraded disposable databases and generate reviewed idempotent SQL	No schema drift or destructive ambiguity	
REG-07	Run CI-equivalent .NET/npm vulnerability and secret scans	No unaccepted blocking advisory or exposed secret	
REG-08	Execute the browser smoke path and changed-feature paths on the exact deployed SHA	UI/API contracts and user-visible states match	
REG-09	Compare the development-story test inventory with current endpoints and this UAT	Every current major feature has a test ID or a documented gap	

API And UI Consistency Checks

1. For every role, compare GET /api/me/access with visible controls and direct API results. The API must independently enforce authorization, and the UI must fail closed when permissions are missing or malformed.
2. Submit identical invalid profile, contract, evidence, task, CMMC, SPR, labor, subcontractor, policy, SSP, report, and administration inputs through UI and API. Server validation is authoritative and messages must not contradict it.
3. Compare list/detail UI values, API values, reports, PDFs, and exports for IDs, counts, status, owner, dates/time zones, sources, classification, review state, and limitations.
4. Exercise loading, empty, success, validation error, authorization denied, server error, refresh, retry, and stale concurrency states on every changed UI surface.
5. Confirm search, filters, sorting, and pagination neither omit eligible current-tenant data nor introduce Tenant B data.

Evidence Capture Requirements

Store evidence under UAT-<release>-<date>/<test-id>/. For each test retain:

- Completed execution record with test ID, actor, role, tenant ID/alias, date/time/time zone, result, defect, and notes.
- Sanitized screenshots or video showing the important UI state and expected result.
- Sanitized request method/path/body, HTTP status, response, correlation/trace ID, and relevant record identifiers.
- Before-and-after state, audit event ID, job/outbox/provider state, and file/report hash where applicable.
- For negative tests, proof that no unauthorized business record, success audit, file, job, notification, token, or external effect was created.
- For Blocked results, the dependency/configuration evidence and named owner. For Not applicable results, the reason and approver.

Redact authorization headers, cookies, tokens, signed URLs, secrets, personal data, and file contents. Never overwrite original failure evidence after a retest.

Defect Severity And Triage

Severity	Definition	UAT disposition
P0	Cross-tenant exposure, authentication bypass, sensitive-data/secret exposure, data loss/corruption, or production-stopping safety failure	Stop testing, contain/escalate, and block all approval
P1	Serious RBAC, audit, No-CUI, upload/scanner, report/export, critical-workflow, or materially misleading-result failure	Block affected release/pilot; fix and run adjacent regression
P2	Important contained functional, accessibility, security-hardening, operational, or test-coverage gap	Accept only with owner, mitigation, expiry, and required approvals
P3	Minor issue without control or workflow impact	Track; does not independently block approval

Every defect must include reproduction steps, role, tenant, endpoint/page, expected and actual results, evidence, No-CUI/security impact, cleanup state, owner, target release, and retest result. Security downgrades require Security approval.

Requirements Traceability Matrix

Requirement	User story or source	Test cases	Expected evidence	Result
Public demo intake, platform follow-up, and customer administration	Demo request/ platform customer code/tests	UAT-M01-M03	Public validation, platform denial, customer list/detail reconciliation, provider result, trace/audit	

Requirement	User story or source	Test cases	Expected evidence	Result
Authentication, invitations, membership, and RBAC	Stories 2.1-3.2; auth/invitation/role tests	UAT-02, UAT-02A, UAT-T01-T04, UAT-I01-I03	Identity, permission array, membership, token lifecycle, denials	
Tenant isolation and subscription lifecycle	Architecture/security invariants; Stories 17.1-17.2	UAT-T01-T04, UAT-I02-I03, all mandatory Tenant B checks	Cross-tenant matrix, lifecycle version/history, no-side-effect proof	
Dashboard and executive readiness	Compliance overview code/tests	UAT-DB01	Reconciled module/API counts, alerts, failure/empty states	
Company profile and external assistance	Stories 7 and 20	UAT-P01-P04, UAT-X03	Draft/completion, source/provenance, read-only/tenant denial	
Contracts, documents, extraction, clauses, applicability, obligations, and content governance	Stories 8-10 and 18-21	UAT-03-UAT-09, UAT-D01-D04, UAT-X01-X04	Source chain, content state, candidate review, obligation/task/evidence linkage, audit	
Tasks, checklists, calendar, notifications, and reminders	Stories 11 and 16; compliance checklist code/tests	UAT-D02-D03, UAT-09, UAT-W01-W03	Cursor, checklist source/progress, dates, assignment, dedupe, outbox/provider result	
Evidence and No-CUI controls	Stories 4, 12, 26; Phase 1A	UAT-04, UAT-10-UAT-12, UAT-E01-E04, UAT-N01	Acknowledgement, classification, scan, private version, block/no-state proof	
CMMC, POA&M, affirmation, responsibility, SPRS, and SSP	Stories 13, 27, 29, 30	UAT-C01-C08	Source/review metadata, immutable calculation/package, closure and share evidence	
Subcontractor and partner workflows	Stories 14, 24, 34	UAT-S01-S04	Profile, flow-down, request, report, portal lifecycle and denials	
SAM.gov SPR preparation	Stories 31.1-31.3	UAT-SPR01-SPR03	Applicability, governed schema, rows, immutable package, receipt/no-submit proof	
Labor readiness	Stories 32.1-32.3	UAT-L01-L03	Source/evidence/task, overlap/redaction/history, dashboard/report prerequisites	
Policy, suggestions, and guarded assistance	Stories 25 and 33	UAT-AI01-AI02	Draft label, citations, classification, human review, safe logs/provider denial	
Reports, exports, and audit	Stories 5 and 15; architecture invariants	UAT-13-UAT-15, UAT-R01-R02	Reconciliation, hashes, lifecycle, authorization, append-only/rollback evidence	
Readiness governance, incident response, and synthetic demo lifecycle	Phase 1A readiness documents and demo dataset tests	UAT-N02-N05	Checklist/evidence/source/history, separation, manifest/scoped cleanup, no-authorization claim	
Enterprise/regulated future boundary	Stories 35-38; roadmap Phase 4	UAT-O01	Disabled or separately governed posture; no certification/authorization claim	
Release regression and UI/API consistency	Regression instructions and automated tests	REG-01-REG-09; API/UI checks	Exact-SHA commands, counts, environments, paired UI/API results	

Test Execution Summary And Sign-Off

Field	Value
Release SHA/tag and deployment ID	
Environment and URLs	
Database, identity, scanner, storage, worker, and provider modes	
Execution start/end and time zone	
Applicable / Passed / Failed / Blocked / Not applicable / Not run	
Open P0 / P1 / P2 / P3 defects	
Regression and migration evidence location	
Synthetic-data cleanup or retention disposition	

Field	Value
Approved exceptions and expiry	
UAT recommendation	Go / Conditional go / No-go

Approver	Name	Decision and date	Scope or limitations	Evidence/ reference
UAT Lead Product Owner Engineering Lead Security Owner Compliance-Content Owner Support/Operations Owner Independent production approver, when applicable				

Known Limitations And Unresolved Gaps

Gap ID	Limitation or inconsistency	Required disposition
GAP-001	Labor dashboard/report contracts, redaction, persistence, repository registration, migration state, and focused tests were reconciled on 2026-09-12. Local focused verification passed; PostgreSQL-backed UAT was not executed by this document update.	Keep UAT-L03 environment-gated until the exact release commit passes CI real-stack tests and staging verification; retain those run links as evidence.
GAP-002	Product strategy describes SSP, SPRS, SPR preparation, labor, full AI, and portals as deferred, while current code, migrations, tests, endpoints, and embedded UI panels implement substantial portions.	Treat them as implemented or partially implemented readiness workflows only; reconcile strategy/roadmap language separately before release.
GAP-003	SAML/SSO/SCIM, government-cloud, FedRAMP, regulated provisioning, enclave, support/emergency access, and customer-managed-key APIs/tests are future enterprise scaffolding.	Keep outside No-CUI MVP acceptance except fail-closed and claim checks; require a separate regulated-release UAT.

Gap ID	Limitation or inconsistency	Required disposition
GAP-004	SAM.gov, Azure Communication Services/ email, HubSpot, external AI, PDF renderer, object storage, scanner, and workers depend on deployed configuration/ providers.	Record provider mode and use approved stubs for workflow tests; never convert unexecuted provider behavior to Pass.
GAP-005	Local development authentication and role selectors do not prove production MFA, logout, token revocation, conditional access, or identity-provider behavior.	Execute identity/session cases in a production-like environment.
GAP-006	Historical launch and pilot evidence predates newer modules and may not match the tested SHA.	Rerun every changed security boundary and attach exact-SHA evidence.
GAP-007	The local database contains accumulated synthetic tenants and duplicate display names.	Select by immutable IDs and a unique run prefix; do not broadly delete records.
GAP-008	/health proves point-in-time reachability, not backup restore, high availability, malware detection, alert delivery, or incident response.	Attach current operational rehearsal and monitoring evidence.
GAP-009	The API inventory is materially broader than the ten primary workspace routes; several workflows are embedded or API-only.	Record UI discoverability gaps separately; endpoint presence is not proof of an executable user workflow.
GAP-010	Compliance sources and SPRS/ SPR rules require current qualified review, publication dates, hashes, and owners.	UAT verifies governance metadata, not substantive legal correctness.
GAP-011	External effects may use an outbox or separate retry boundary rather than the relational request transaction.	Test provider failures and document the exact atomicity and duplicate-delivery limits.

Recommended missing automation: generate an endpoint-to-permission-to-UAT inventory; add browser role/state coverage for embedded panels; run PostgreSQL concurrency and audit-rollback tests for every new lifecycle; inject worker/provider failures; scan exports for Tenant B markers, formulas, prohibited classifications, and missing disclaimers; and automate production-like session expiration/revocation.

Hidden Risks, Edge Cases, And Dependencies

Item	Risk
Platform authorization	Tenant onboarding is internal platform operations. Owner, Admin, and ManageTenant do not imply ProvisionTenants; staging and production require the controlled Gccs . PlatformOperator application role.
Local platform permission	Local DefaultPlatformPermissions is empty. Start Vite with VITE_GCCS_DEV_PLATFORM_PERMISSIONS=ProvisionTenants; an already-running web process will not acquire the variable until restarted.
Unique onboarding references	Customer and subscription references are duplicate-protected. Increment the synthetic suffix for each rerun instead of editing or deleting prior onboarding records.

Item	Risk
Invitation delivery	Azure Communication Services, sender-domain configuration, managed-identity permission, and worker settings are external dependencies. Queued proves pending creation, not email receipt or Owner activation.
Paid billing lifecycle	FeDril stores an operator-confirmed plan and subscription reference but does not verify payment or automate renewal, delinquency, cancellation, suspension, or archival. Do not claim billing-system enforcement.
Invitation identity	The invitation token is single-use and not exposed on the platform result. Activation requires the exact invited email; do not retrieve tokens from storage or logs for UAT convenience.
Pending cancellation	Cancellation is limited to <code>PendingOwnerAcceptance</code> plus <code>PendingActivation</code> . Activated Pilot and Paid tenants require a separate lifecycle process and cannot be cancelled through the pending-onboarding action.
Local development auth	Role switching may require API header changes or manual test setup; the default UI does not present a production sign-in flow.
Existing profile data	A tenant has one current company profile. Run these steps in a dedicated UAT tenant so the synthetic profile does not overwrite customer metadata.
Profile completion	100% reflects presence of the implemented profile fields. It is not verification of SAM registration, SBA size status, eligibility, certification, or legal compliance.
SAM.gov lookup	Profile completion does not require the optional SAM.gov lookup. Provider availability and lookup correctness are outside this focused UAT unless separately tested.
Static test dates	The supplied dates support this fixed fixture. Update contract, deliverable, and calendar date ranges together if the UAT is executed after the fixture period.
Deliverable cleanup	The current API exposes list, create, and update operations but no deliverable deletion operation. Use a disposable UAT tenant or retain synthetic records as test evidence.
Audit failure atomicity	Focused tests prove successful profile and deliverable audit creation, but do not prove rollback under an injected audit-writer failure. Do not claim mutation-and-audit atomicity without that additional test.
Clause IDs	Clause search returns implementation-specific IDs; testers should copy the actual published IDs from the current environment.
Obligation generation timing	Some obligations may generate automatically on clause attachment; others may require an explicit generate action.
Evidence linking	Evidence can be accepted without an obligation link if the obligation ID is not visible; this reduces report package completeness.
CMMC readiness scope	CMMC module results are readiness workflow records only. They do not prove CMMC certification, assessment success, legal compliance, or authorization to store real CUI.
CMMC controls	Available controls depend on the seeded control library and selected assessment level. A Level 1 assessment will not show every Level 2 control.
CMMC POA&M dates	The fixed POA&M due date is intentionally synthetic. If the date is in the past when UAT runs, the item may display <code>Overdue</code> ; that is acceptable for this fixture.
Report scope	Evidence packages include only evidence matching selected scope and status rules. Draft/rejected evidence requires authorization.
Audit filters	Audit records may be easier to find by <code>Entity</code> than by exact actor when local development uses generated user IDs.
No-CUI enforcement	Do not use real CUI to test blocking. Use synthetic classification labels and allowed synthetic text only.
Customer-facing claims	This UAT proves workflow behavior only. It does not prove CMMC certification, legal compliance, government approval, or production CUI readiness.
Cross-tenant cache and pagination	Tenant switching, cached detail panels, signed cursors, exports, and background jobs must all re-resolve the active tenant. A correct direct record lookup does not prove aggregate isolation.
Embedded and API-only workflows	Several newer features appear inside Contracts, CMMC, Evidence, or Settings rather than as primary routes; some are API-only. Testers may need an API harness, and lack of discoverability must be recorded separately.
Scanner terminology	Local health currently identifies a configured development ClamAV placeholder. Reachability is not proof of detection quality, quarantine, signature freshness, fail-closed behavior, or production configuration.
File replacement and cleanup	A rejected upload must not leave an object, version pointer, extraction job, or cleanup backlog that exposes content. Verify object and cleanup state, not only the UI response.

Item	Risk
External provider atomicity	Email, CRM, lookup, and other provider calls may be outside the relational transaction. Verify outbox/idempotency behavior and document duplicate-delivery or partial-failure windows.
SPR vocabulary	
Labor implementation state	Canonical product behavior is SAM.gov Subcontracting Plan Reporting preparation. Legacy eSRS identifiers remain internal compatibility names and must not be presented as a current external submission integration.
Future enterprise APIs	The reconciled labor report/dashboard implementation passes focused local compilation, tests, and EF migration-drift validation. Do not infer deployed behavior until exact-commit CI and staging evidence pass.
Compliance source freshness	SAML/SCIM, government-cloud, FedRAMP, enclave, customer-managed-key, support-access, and emergency-access code does not prove deployed provider configuration, assessed controls, or authorization.
Fixed original dates	A source URL alone is insufficient. Current publication state, review owner, effective date, last-reviewed date, confidence, and source hash/version must be captured for customer-visible content and calculations.
	Original PDF fixtures intentionally contain historical dates for overdue checks. Preserve that expectation or update every related contract, task, calendar, POA&M, and report assertion together.

Document Change History

Version	Date	Change
1.0	2026-06 historical	Original PDF-aligned No-CUI onboarding-to-report UAT.
2.0	2026-09-12	Preserved the original PDF format and 32 original test cases; added 42 executable cases for current application features, expanded synthetic fixtures, security/negative tests, smoke and regression suites, API/UI consistency checks, evidence requirements, defect triage, traceability, gaps, sign-off, and per-case execution records.